

Trento Smart Mountain

Milestone #4 – Closing the Sprint!

Gruppo: ID – 6

Componenti: Federico Cattelan – 242111

Marco Christian Stoica – 246443

Giacomo Radin – 242907



Scadenza: 10/06/2026

Anno Accademico 2025/2026

“Il vero viaggio di scoperta non consiste nel cercare nuove terre, ma nell’avere nuovi occhi.”



Abstract Sprint 2

Il documento di Milestone #4 chiude il ciclo Agile SCRUM di Trento Smart Mountain, formalizzando le dinamiche del team durante lo *Sprint 2*. Lo sprint si è concentrato su:

- (i) il **profilo utente v2 con onboarding 3-step**, comprensivo di un meccanismo *anti-cheat lato server* su birthDate e caiLevel;
- (ii) il **security hardening** completo del backend (rate limit a 6 livelli, refresh token rotation con replay detection, mappatura globale degli errori di business);
- (iii) la **robustezza del sync mobile** (Room WAL v8, retry incrementale, login email/username, UI premium glass/aurora);
- (iv) il **Social Feed completo** con post, like, commenti, follow/unfollow e sistema di **Storie 24h** con editor visivo drag&drop (mappa, polyline, sticker, font), condivisione sessioni pianificate e attività post-hike;
- (v) il **redesign architetturale del ciclo di vita sessione** (ADR-001): participationState state machine, approvazione partecipanti, force-close capogruppo con auto-finalizzazione;
- (vi) la **copertura test** portata da 0 a **262/262 test Jest verdi** (19 suite).

Il documento dettaglia inoltre i 3 bug critici scoperti nell’audit di fine sprint (discriminator persistence, anti-cheat bypassabile, JWT expiry insufficiente per uso offline) e il loro fix in-sprint, a riprova di un processo Agile maturo basato sull’auto-critica continua.

Indice

1	Sezione Introduttiva	3
2	Sezione Generale	5
2.1	Strategia di Branching	5
2.2	Product Backlog	7
2.3	Definizione di “Done”	12
3	Sezione Sprint #2	13
3.1	Goal	13
3.2	Sprint Planning (Sprint Backlog)	13
3.3	Test Cases	22
3.4	Sprint Review	26
3.5	Product Backlog Refinement	27
3.6	Sprint Retrospective	29
4	Visione Sprint 3	33
4.1	Obiettivi e roadmap	33
5	Sezione Finale	36
5.1	Diagramma del Deploy Finale	36
5.2	Stack Tecnologico	37
5.3	Conclusioni	39
A	- API implementate Sprint 1 + Sprint 2	40
B	Utilizzo Intelligenza Artificiale nello Sviluppo	46
C	Schermate Android	47
C.1	Nuova palette e pivot grafico	47

1 Sezione Introduttiva

Team Members

Nome	Cognome	Matricola	Ruolo Agile	Account GitHub
Federico	Cattelan	242111	SCRUM Master	@federicoca
Marco Christian	Stoica	246443	Developer	@STUSSY-user
Giacomo	Radin	242907	Developer	@giacomoradin

Tabella 1: Componenti del team di sviluppo con ruoli Agile SCRUM.

Tutti e tre i componenti hanno contribuito con commit al repository condiviso, come verificabile dalla cronologia GitHub.

Project Idea

Trento Smart Mountain (TSM) è un ecosistema digitale per l'escursionismo in Trentino-Alto Adige che integra **sicurezza attiva di gruppo** (tracciamento GPS in background con Write-Ahead Log per crash-safety, codici invito sessione, fallback SOS via beacon BLE), **gamification educativa** (modello CAI di stima sforzo, Social Credits cumulativi, quiz formativi, check-in NFC ai totem di vetta) e **gestione rifugi** (account dedicati, dashboard telemetria IoT, modulo rifiuti & logistica). Il sistema combina un'app Android nativa (Kotlin 2.2 / Jetpack Compose, MVVM, offline-first) con un backend Node.js + MongoDB Atlas (indici geospaziali 2dsphere, JWT con refresh rotation, anti-cheat server-side) e un'integrazione meteo reale (TINIA / meteo.report) per supportare l'escursionista dalla pianificazione fino al ritorno a valle.

Links

- **Repository GitHub:** https://github.com/giacomoradin/Trento_Smart_Mountain
- **Documentazione API (Apiary/SwaggerHub) – Sprint 1+2:** <https://app.swaggerhub.com/apis/unitn-1d6/trento-smart-mountain-api/1.0.0>
- **Swagger UI live (deploy Render):** <https://trento-smart-mountain-xz7u.onrender.com/api-docs>
- **Deploy backend (Render Free tier):** <https://trento-smart-mountain-xz7u.onrender.com>
Nota: cold start ~30–100 s alla prima request dopo inattività, dovuto al free tier di Render.
- **APK Android (download diretto):**
[Google Drive – TSM v2.0 APK debug](#)
(APK debug: firmato con chiave dev.)

Istruzioni di installazione (Android sideload)

1. Scaricare il file TSM-debug.apk dal link Google Drive sopra riportato.
2. Sul dispositivo Android (**Android 9.0+, API 28+**): aprire Impostazioni → Sicurezza → Installa app sconosciute e abilitare per il browser/gestore file utilizzato.
3. Aprire il file .apk scaricato e confermare l'installazione.
4. Al primo avvio: registrare un account o usare le credenziali demo riportate nella tabella sottostante.
5. Assicurarsi di avere GPS abilitato (obbligatorio per tracking attività) e connessione internet per il primo accesso.



Nota: Il backend su Render Free Tier può impiegare 30–100 s per la prima richiesta dopo inattività (cold start). L'app gestisce automaticamente il retry con feedback visivo.

Account demo per classe di utenza

Ruolo	Email	Username	Password
Hiker	fedeti3312@fixscal.com	DemoHiker	DemoHiker2026!
Refuge	sagoje7419@fixscal.com	DemoRifugio	DemoRifugio2026!

Tabella 2: Account demo per ciascuna classe di utenza (un account per ruolo, come richiesto dal template).

Gli account demo sono pre-popolati al primo boot del backend Render tramite seed idempotente; restano protetti dal medesimo stack di sicurezza degli account reali (Joi validation, rate limit, refresh token rotation).

2 Sezione Generale

2.1 Strategia di Branching

Il team conferma la strategia *Git Flow semplificata* adottata in Sprint 1, evitando rigorosamente la logica “*Master only strategy*”. Rispetto allo Sprint 1, in Sprint 2 sono state introdotte le seguenti **variazioni**:¹

- **Auto-deploy su Render:** il branch UI (e i suoi merge) è ora collegato al deploy automatico su Render Free tier, abilitando una validazione per “*mockare*” il deploy di un hosting su server AWS.

Branch attivi e storici (aggiornamento Sprint 2)

Branch	Scopo	Stato
main	Release stabile; solo merge da PR approvate. Nessun push diretto.	● Attivo
develop	Branch di implementazione Sprint 1: convergenza tra feature mobile e API.	✓ Mergiato
UI	Branch di integrazione Sprint 2 (mobile + backend). Auto-deploy su Render.	✓ Mergiato
US47-Sentieri	Feature branch per integrazione sentieri (sprint 2 mobile).	✓ Mergiato
implementazione-security-best-practise-e-fix-bug-da-sprint-1	Profilo utente con personalInfo, experience, preferences, weeklyGoals; onboarding 3-step skippable.	✓ Mergiato
Debug	Branch nato appositamente per fare debugging ma non è stato più utilizzato. Obsoleto.	● Obsoleto
Testing-API-Jest	Access + refresh token (TTL 15m / 30d), replay detection con revoca famiglia, TsmAuthenticator OkHttp lato mobile.	✓ Mergiato
US-7-generazione-checklist	Branch dedicato per lo sviluppo della user story 7.	✓ Mergiato
US-20-14-LiveTracking	Branch dedicato per lo sviluppo delle user stories 20-14.	✓ Mergiato
US-11-22-21-SOS	Branch dedicato per lo sviluppo delle user stories dedicate alla gestione dell'SOS: 11-22-21.	✓ Mergiato

Tabella 3: Strategia di branching aggiornata per Sprint 2 (GitFlow semplificato esteso con i pattern feat/_ e fix/_). I branch merged non vengono eliminati per consentire la verifica dei docenti.

¹Alcuni commit presenti nel repository (STUSSY-user) non contengono codice applicativo ma file GPX e KML utilizzati come hosting statico per i dati geografici (sentieri, tracciati) consumati dall'app Android in sostituzione di chiamate a API esterne.

Convenzioni operative confermate

1. Una branch per gestire Issue GitHub.
2. PR obbligatoria verso develop (Sprint 2) in caso di revisione di codice già presente o in caso di merge su main – mai push diretto su main.
3. Commit semantici: feat:, fix:, refactor:, docs:, chore:.
4. Merge tramite Pull Request con revisione esplicita di almeno un altro membro del team.

❗ Nota per i docenti: I branch **non vengono cancellati** dopo il merge, in conformità con le indicazioni del docente, per permettere la verifica completa della storia di sviluppo nel repository GitHub.

2.2 Product Backlog

Di seguito il Product Backlog aggiornato a fine Sprint 2 (10/06/2026).

ID	Attore / Ruolo	Nome User Story	User Story	Imp.
56	Sistema	Risoluzione bug di sistema	Come sistema, voglio correggere i bug emersi in Sprint 1 e durante Sprint 2, così da garantire stabilità in demo e su Render.	3
48	Sistema	Sicurezza di sistema	Come sistema, voglio che l'applicazione garantisca la protezione dei dati, il controllo degli accessi e la prevenzione di utilizzi non autorizzati, in modo da ridurre i rischi di sicurezza, assicurare l'integrità delle informazioni e rispettare i requisiti normativi.	4
2	Tutti gli utenti	Accesso offline con token JWT	Come utente già autenticato, voglio poter accedere all'app anche senza connessione internet, così da usare le funzionalità anche in montagna.	5
7	Partecipante	Checklist equipaggiamento dinamica	Come escursionista voglio ricevere una checklist dell'equipaggiamento necessaria basata sull'itinerario e sulle condizioni meteo (durante i giorni precedenti alla partenza), così da prepararmi adeguatamente	15
12	Partecipante	Mappa offline e bussola	Come escursionista, voglio poter consultare una mappa offline con la mia posizione e una bussola così da orientarmi anche senza copertura internet	20
10	Partecipante	Tracciamento GPS in background	voglio che la mia posizione GPS venga tracciata automaticamente in background durante l'escursione, così da garantire la mia sicurezza e quella del gruppo	22
13	Partecipante	Auto-Pause GPS da fermo	escursionista, voglio che il tracciamento GPS si riduca automaticamente quando sono fermo, così da risparmiare la batteria del dispositivo	24
11	Partecipante	Invio segnale SOS	Come escursionista in difficoltà voglio poter inviare un segnale SOS con le mie coordinate GPS così da ricevere soccorso il prima possibile.	28
34	Sistema	Risposta API < 500ms (p95)	Come sistema voglio che le query vengano elaborate velocemente così da garantire un'esperienza utente fluida	35

continua...

ID	Attore / Ruolo	Nome User Story	User Story	Imp.
47	Tutti gli utenti	Planning	Come utente voglio poter selezionare il punto di arrivo e di partenza in modo tale da visualizzare i sentieri percorribili	37
22	Capogruppo	Ricezione e validazione SOS	Come Capogruppo, voglio ricevere i segnali SOS dei partecipanti e poterli validare tramite la mia dashboard, così da attivare i soccorsi solo in caso di reale necessità.	38
39	Sistema	Download preventivo Hike Packet (offline-ready)	Come sistema, voglio scaricare preventivamente la traccia GeoJSON e le Map Tiles OSM con padding di 1 km, così da garantire la navigazione offline in zona di not-spot.	42
20	Capogruppo	Dashboard tracking GPS in tempo reale	Come Capogruppo, voglio poter visualizzare la posizione GPS di tutti i partecipanti in tempo reale sulla mappa, così da monitorare la coesione del gruppo. Il tutto deve avere un'interfaccia pulita e intuitiva.	45
21	Capogruppo	Gestione emergenze e broadcast allarmi	Come Capogruppo, voglio poter ricevere un allarme dai componenti del gruppo (online e beacon BLE) così da gestire le emergenze e coordinare la risposta.	52
55	Utente	Interfaccia utente e grafica	Come utente, voglio un'interfaccia coerente (palette, tipografia, icone) in tutta l'app, così da avere un'esperienza professionale e leggibile.	60
14	Partecipante	Visualizzazione posizione relativa al gruppo	Come escursionista, voglio vedere la posizione degli altri membri del gruppo sulla mappa, così da sapere dove si trovano rispetto a me, con un'implementazione efficiente ed efficace anche a livello energetico.	62
37	Sistema	Sincronizzazione batch Event Sourcing	Come sistema, voglio sincronizzare gli eventi di gamification accumulati offline tramite Event Sourcing in batch, così da garantire la consistenza del saldo Crediti Sociali.	64
54	Utente	Sicurezza del profilo personale in app	Come utente, voglio verificare la password prima di modificare dati sensibili dell'account e poterla cambiare in sicurezza, così da proteggere le mie informazioni.	68
continua...				

ID	Attore / Ruolo	Nome User Story	User Story	Imp.
26	Escursionista	Annullamento segnale SOS	Come escursionista che ha premuto SOS per errore, voglio poter annullare il segnale prima che venga inoltrato, così da evitare l'attivazione inutile dei soccorsi.	69
38	Sistema	Idempotenza transazioni crediti (UUID v4)	Come sistema, voglio garantire che ogni evento di gamification sia processato una sola volta, così da prevenire il double-spending dei Crediti Sociali. Idempotenza	72
17	Partecipante	Ricezione alert meteo e pericoli	Come escursionista, voglio ricevere notifiche di allerta meteo o pericoli nel percorso, così da prendere decisioni informate sulla mia sicurezza.	76
27	Gestore Rifugio	Invio allerta pericolo push	Come Gestore Rifugio, voglio poter inviare notifiche push di allerta pericolo agli escursionisti nella zona, così da informarli tempestivamente di situazioni pericolose.	80
43	Sistema	Timeout API Meteo max 3 secondi	Come sistema, voglio che il polling alle API di MeteoTrentino abbia un timeout massimo di 3 secondi, così da non bloccare l'utente in caso di lentezza del servizio esterno.	84
35	Sistema	Storage locale SQ-Lite max 50MB con FIFO	Come sistema, voglio che il database locale non superi 50 MB applicando una politica FIFO, così da non saturare lo storage del dispositivo mobile.	88
49	Utente	Schermata Profilo	Come utente, voglio una schermata Profilo completa (dati personali, esperienza, obiettivi, livello e crediti sociali, foto), così da gestire la mia identità in app e vedere la mia progressione.	92
50	Utente	Schermata Feed	Come utente, voglio feed sociale, interazioni (follow, like, commenti), stories 24h e editor visivo, così da seguire la community e condividere le escursioni.	96
16	Partecipante	Consultazione dati di navigazione	Come escursionista, voglio poter consultare i dati di navigazione (quota, velocità, distanza percorsa) durante l'escursione, così da monitorare il mio progresso.	100
continua...				

ID	Attore / Ruolo	Nome User Story	User Story	Imp.
41	Sistema	Compatibilità Android 9.0+ (API 28)	Come sistema, voglio essere compatibile con dispositivi Android 9.0 o superiori, così da coprire la maggior parte del parco dispositivi degli utenti.	101
52	Utente	Modulo formazione (Quiz)	Come utente, voglio il modulo Formazione con quiz a risposta multipla, crediti al primo superamento e checkpoint NFC (mockup), così da imparare e progredire nei livelli.	103
30	Gestore Rifugio	Monitoraggio affollamento rifugio	Come Gestore Rifugio, voglio poter visualizzare il conteggio delle persone presenti nel rifugio tramite sensori ottici, così da gestire la capacità in modo efficiente.	104
45	Partecipante	Visualizzazione difficoltà e dislivello	Come escursionista, voglio vedere la difficoltà tecnica e il dislivello di una attività, così da valutare se è adatto alle mie capacità in modo chiaro sull'interfaccia grafica in app	108
44	Tutti gli utenti	Recupero password	Come utente che ha dimenticato la password, voglio poter richiedere il reset tramite email, così da riacquisire accesso al mio account.	112
46	Partecipante	Abbandono sessione escursione	Come escursionista, voglio poter abbandonare una sessione di gruppo, così da gestire autonomamente la mia partecipazione.	123
8	Partecipante	Reindirizzamento store partner	Come escursionista, voglio poter essere reindirizzato agli store partner per acquistare l'attrezzatura mancante, così da completare il mio equipaggiamento comodamente.	130
24	Capogruppo	Failover automatico leadership	Come partecipante, voglio che in caso di indisponibilità del Capogruppo per 10+ minuti, venga eletto automaticamente un nuovo leader, così da garantire la continuità della gestione delle emergenze.	135
25	Capogruppo	Risoluzione conflitto doppia leadership (anti-split-brain)	Come sistema, voglio risolvere automaticamente i conflitti quando due partizioni mesh hanno leader distinti, così da garantire un'unica leadership coerente al ricongiungimento.	141
28	Gestore Rifugio	Monitoraggio telemetria macchinari IoT	Come Gestore Rifugio, voglio poter monitorare lo stato dei macchinari (compattatori, disidratatori) tramite dashboard, così da gestire la manutenzione preventiva.	154
continua...				

ID	Attore / Ruolo	Nome User Story	User Story	Imp.
29	Gestore Rifugio	Reset macchinari IoT	Come Gestore Rifugio, voglio poter resettare i macchinari IoT del rifugio tramite l'app, così da risolvere malfunzionamenti senza intervento tecnico in loco.	158
31	Amministratore	Gestione promozioni partner	Come Amministratore, voglio poter inserire e gestire le promozioni dei partner commerciali, così da offrire sconti agli utenti che spendono i loro Crediti Sociali.	165

Tabella 4: Product Backlog attivo Sprint 2 con priorità (Importanza).

Legenda variazioni al Product Backlog (Sprint 2)

A fine Sprint 2 il backlog ha subito le seguenti variazioni rispetto al baseline di Sprint 1 (evidenziate nel file Excel, presente nella repository github:Backlog V1 - Active Product Backlog):

- **Nuove User Story Sprint 2 (evidenziate in verde):** US-47 (Planning), US-48 (Sicurezza sistema), US-49 (Profilo v2), US-50 (Social Feed), US-52 (Quiz backend), US-54 (Sicurezza profilo anti-cheat), US-55 (UI glass/aurora), US-56 (Bug fix batch). Storie nate dall'analisi in sprint e integrate nel backlog corrente.
- **Debito tecnico Sprint 3 (evidenziate in blu):** US-17 (Alert meteo push), US-27 (Notifica pericolo rifugio), US-30 (Monitoraggio affollamento IoT). Effort residuo complessivo: **20 SP**. Non completabili in Sprint 2 per dipendenze hardware (gateway MQTT, sensori IoT fisici, BLE Mesh); traslate a Sprint 3 con priorità Must.
- **Rimosse dal backlog attivo (evidenziate in rosso):** US-23 (Relay CNSAS), US-36 (AES-256 BLE), US-40 (TTL firma ECC), US-42 (Foreground Service isolato). Descope per complessità hardware fuori perimetro universitario (US-23, US-36, US-40) o già integrata in altro (US-42 assorbita nella gestione sessione ADR-001).

2.3 Definizione di “Done”

La definizione di Done adottata in Sprint 1 è stata **estesa in Sprint 2** per riflettere la maggiore maturità del processo (in particolare l'introduzione di test automatici e di un audit di sicurezza prima della chiusura sprint). Una User Story è dichiarata **Done** se soddisfa *tutti* i seguenti criteri:

1. **Completamento funzionale:** Il comportamento descritto è implementato ed è verificabile sul branch UI (auto-deploy su Render).
2. **Qualità del codice:** Commenti dove necessario (logica non ovvia, scelte architetturali, edge case); aderenza alle convenzioni MVVM + 3-layer route→service→model.
3. **Integrazione nel repository:** Mergiato tramite PR su UI; nessun conflitto irrisolto.
4. **Build e stabilità:** Backend `npm run dev` parte senza eccezioni; mobile `./gradlew compileDebugKotlin` e `./gradlew assembleDebug` verdi.
5. ★ (Nuovo in Sprint 2) **Test automatici:** Ogni endpoint backend pubblico ha almeno un test Jest associato (caso di successo + almeno un caso di errore); regressione bloccante.
6. ★ (Nuovo in Sprint 2) **Audit di sicurezza a 2 passi:** Analisi statica (CWE-classified) sul codice modificato + cross-validation post-fix per intercettare regressioni di sicurezza (vedi CLAUDE.md §5).
7. **Verifica manuale:** Smoke test del flusso principale; bug evidenti al primo utilizzo invalidano lo stato di Done.
8. **Tracciabilità:** La story è aggiornata nello strumento di sprint (Sprint 2 Backlog.xlsx).
9. **Documentazione leggera:** Se introduce setup nuovo o env vars, aggiornare `docs/setup_.md` o `TSM_PROJECT_STATE.md`.
10. **API contract:** Ogni nuovo endpoint è descritto in Swagger (`swagger-output.json`); ogni endpoint che tocca dati utente è protetto da `authenticate` + (se necessario) `requireRoles`.
11. ★ (Nuovo in Sprint 2) **Discriminator alignment:** Ogni write su campi del sotto-schema Hiker/Refuge/Admin usa esplicitamente il modello del discriminator, mai `User.findByIdAndUpdate`.

❗ **Lezione appresa dallo Sprint 1:** I 3 bug critici scoperti nell'audit interno di Sprint 1 (C1 partecipante non-creator, C2 endpoint Weather non protetti, C3 ACCESS_BACKGROUND_LOCATION mancante) hanno motivato l'introduzione dei criteri 5, 6 e 11 al fine di intercettare *strutturalmente* e non occasionalmente questa classe di errori. In Sprint 2 il criterio 11 ha permesso di scoprire **prima del rilascio** il bug “discriminator persistence” che altrimenti sarebbe rimasto silente in produzione.

3 Sezione Sprint #2

3.1 Goal

*“Trasformare TSM da MVP funzionale a piattaforma completa e robusta:
 (i) profilo utente v2 con onboarding 3-step e anti-cheat server-side sui campi di scoring;
 (ii) security hardening OWASP API (rate limit 6 livelli, Joi, refresh token rotation con replay detection, global error mapper);
 (iii) Social Feed completo + sistema Storie 24h con editor visivo drag&drop (mappa, polyline, sticker, font);
 (iv) redesign architetturale del ciclo di vita sessione ADR-001 (state machine participationState, approvazione partecipanti, force-close capogruppo);
 (v) robustezza mobile (Room WAL v8, retry incrementale, login email o username);
 (vi) portare la copertura test da 0 agli effettivi **262 test Jest verdi** (su 19 suite) implementati.”*

3.2 Sprint Planning (Sprint Backlog)

Parametri dello Sprint

Parametro	Valore
Durata	~3 settimane (18/05/2026 – 10/06/2026)
Capacità team	3 membri $\times$ ~72 h/settimana = 216 h/settimana $\times$ 3 settimane $\approx$ 650 h totali
Story points pianificati	78 (SS1: 24, SS2: 28, SS3: 26)
Story points completati	71 (91%)
Effort task-level pianificato	409 unità (stime per task, Sprint 2 Backlog)
Effort task-level completato	389 unità
Debito tecnico Sprint 3	7 SP $\approx$ 20 unità effort (US-17, US-27, US-30 — dipendenze hardware/IoT)

Tabella 5: Parametri generali dello Sprint 2.

Andamento dello sprint

Settimana	SP Pianificati	SP Completati	Note principali
SS1 (18/05–24/05)	24	26	Profilo v2 + onboarding 3-step; ProfileViewScreen read-only; auto-seed quiz; batch fix critico 26/05 (discriminator persistence + anti-cheat server-side); 78/78 test verdi.
SS2 (25/05–31/05)	28	28	Social Feed backend (Post, Feed, Follow, Like, Commenti); sistema Storie 24h (entità reale, TTL 24h, media Base64); ADR-001 session lifecycle (participationState state machine, approvazione/rifiuto, force-close); Emergency SOS backend; global error mapper (50+ codici); meetingDate migration; refresh token rotation; TsmAuthenticator OkHttp.
SS3 (01/06–10/06)	26	17	Storie mobile (composer drag&drop, viewer multi-segmento con Video-View); approvazione/rimozione partecipanti; login email/username; UI premium (glass morphism, aurora particles, frecce animate GPX); mappa GPX attività condivise; fix crash/schermate bianche; Room WAL (migrations v5→v8) + retry; 262/262 test verdi (19 suite). Debito tecnico 20 effort: US-17, US-27, US-30 (dipendenze hardware/IoT).

Tabella 6: Distribuzione degli story point pianificati e completati nelle settimane dello Sprint 2.

$SS_n \rightarrow$ sub-sprint settimanale.

Sprint Backlog

				Estimated Effort Remaining																		
ID Item	Sprint Task	Vol.	Init.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	
48	Come sistema, voglio che l'applicazione garantisca la protezione dei dati, il controllo degli accessi e la prevenzione di utilizzi non autorizzati, in modo da ridurre i rischi di sicurezza, assicurare l'integrità delle informazioni e rispettare i requisiti normativi.	1. Limitazione della frequenza delle richieste su endpoint pubblici (basato su IP e utente)	Giacomo	3	3	2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	
		2. Convalida e sanificazione di tutti gli input utente	Giacomo	3	3	2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		3. Gestione sicura delle API (rimozione delle chiavi hard-coded, rotazione delle chiavi JWT, seguire le best practices OWASP senza intaccare le funzionalità esistenti	Giacomo	3	3	2	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		4. Testing	Giacomo	2	2	2	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		5. Documentazione	Giacomo	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		6. Deploy	Giacomo	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
47	Come utente voglio poter selezionare il punto di arrivo e di partenza in modo tale da visualizzare i sentieri percorribili	1. UI pianificazione sessione da sentieri su mappa da database.	Federico	3	3	3	3	3	3	2	2	1	1	-	-	-	-	-	-	-	-	
		3. Modello sentiero e metodi CRUD necessari solo (get, e get(id))	Marco	2	2	2	2	2	2	2	1	1	-	-	-	-	-	-	-	-	-	-
		3. Modello Punto di partenza/arrivo e metodi CRUD	Marco	3	3	3	3	3	3	2	2	1	-	-	-	-	-	-	-	-	-	-
		4. Logica associazione Punto meta /partenza e sentieri relativi	Marco	3	3	3	3	3	3	3	2	2	1	-	-	-	-	-	-	-	-	-
		5. Endpoint/query sentieri percorribili tra due punti GeoJSON.	Marco	4	4	4	4	4	4	4	3	2	1	-	-	-	-	-	-	-	-	-
		6. Preview percorso su mappa in tab Pianifica.	Federico	3	3	3	3	3	3	3	3	3	2	1	-	-	-	-	-	-	-	-
		7. Testing	Marco	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-
		8. Documentazione	Marco	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-
		9. Deploy	Federico	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-
7	Come escursionista voglio ricevere una checklist dell'equipaggiamento necessaria basata sull'itinerario e sulle condizioni meteo (durante i giorni precedenti alla partenza), così da prepararmi adeguatamente	1. Integrazione API meteo TINIA/meteo.report con persistenza MongoDB.	Marco	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	
		2. Modello Location GeoJSON e endpoint forecast 3h/24h.	Marco	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		3. Integrazione meteo reale in SessionDetailScreen.	Giacomo	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		4. Sviluppo UI checklist con drag-and-drop e spunta.	Giacomo	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		5. Sviluppo algoritmo di mappatura meteo/altitudine → equipaggiamento.	Marco	4	4	4	4	4	4	3	2	-	-	-	-	-	-	-	-	-	-	-
		6. Testing API	Marco	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-
		7. Documentazione	Marco	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-
		8. Deploy	Federico	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-
		12	Come escursionista, voglio poter consultare una mappa offline con la mia posizione e una bussola così da orientarmi anche senza copertura internet	1. Integrazione OSMDroid per rendering mappa e marker utente.	Federico	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
2. UserLocationTracker e indicatore segnale GPS.	Federico			0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
3. Centratura mappa e lifecycle TsmMapView.	Federico			1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
4. Lettura SensorManager Accelerometro	Federico			0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
6. Download e cache Map Tiles OSM offline.	Federico			4	2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
7. Testing	Federico			1	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
8. Documentazione	Federico			1	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
9. Deploy	Federico			1	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

continua...

ID Item	Sprint Task	Vol.	Init.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
10	voglio che la mia posizione GPS venga tracciata automaticamente in background durante l'escursione, così da garantire la mia sicurezza e quella del gruppo	1. Integrazione FusedLocationProviderClient.	Federico	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		2. HikeTrackingEngine e visualizzazione traccia su mappa.	Federico	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		3. Sviluppo logica di caching locale offline (Room/SQLite telemetria).	Federico	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		4. Worker periodico per invio batch coordinate al server.	Marco	3	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		5. Testing	Federico	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		6. Documentazione	Giacomo	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		7. Deploy	Federico	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
13	escursionista, voglio che il tracciamento GPS si riduca automaticamente quando sono fermo, così da risparmiare la batteria del dispositivo	1. StationaryDetector e logica auto-pause.	Federico	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		2. UI notifiche e metriche durante auto-pausa.	Federico	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		3. Integrazione Activity Recognition API.	Federico	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		4. Testing	Federico	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		5. Documentazione	Giacomo	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		6. Deploy	Federico	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
42	Come sistema voglio mantenere attivo tracciamento GPS tramite Foreground Services così da evitare che Android interrompa i processi (Doze Mode)	1. Implementazione Android Foreground Service con notifica persistente.	Federico	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		2. Richiesta permessi a runtime (Location).	Federico	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		3. Configurazione WakeLock e ottimizzazione Doze Mode.	Federico	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		4. Test del ciclo di vita del servizio in background.	Federico	3	3	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		5. Documentazione	Federico	1	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		6. Deploy	Federico	1	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
11	Come escursionista in difficoltà voglio poter inviare un segnale SOS con le mie coordinate GPS così da ricevere soccorso il prima possibile.	1. UI pulsante SOS (dialog conferma in RegistraScreen).	Giacomo	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		2. Beacon BLE TSM + POST HTTPS	Federico	3	3	3	3	3	3	2	-	-	-	-	-	-	-	-	-	-	-
		3. Endpoint POST /api/v1/emergencies backend.	Federico	2	2	2	2	2	2	-	-	-	-	-	-	-	-	-	-	-	-
		4. Dialog attivazione Bluetooth / invio senza beacon	Federico	2	2	2	2	2	2	2	-	-	-	-	-	-	-	-	-	-	-
		5. Gestione accodamento invio in assenza di rete.	Federico	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-
		6. Testing	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-
		7. Documentazione	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-
		8. Deploy	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-
2	Come utente già autenticato, voglio poter accedere all'app anche senza connessione internet, così da usare le funzionalità anche in montagna.	1. Caching stato autenticazione e token validi in EncryptedSharedPreferences.	Federico	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		2. Ripristino sessione JWT all'avvio app (AuthSession).	Federico	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		3. Interceptor HTTP e gestione errori offline.	Federico	1	2	2	2	2	-	-	-	-	-	-	-	-	-	-	-	-	-
		4. Cache profilo utente con Room.	Federico	1	2	2	2	2	-	-	-	-	-	-	-	-	-	-	-	-	-
		5. Gestione state-machine dell'app offline/online.	Giacomo	2	2	2	2	2	-	-	-	-	-	-	-	-	-	-	-	-	-
		6. Testing	Giacomo	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-
		7. Documentazione	Federico	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-
		8. Deploy	Federico	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-
34	Come sistema voglio che le query vengano elaborate velocemente così da garantire un'esperienza utente fluida	1. Creazione indici spaziali MongoDB (sessions/locations).	Marco	0	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		2. Ottimizzazione query delle coordinate.	Marco	4	4	4	4	4	4	4	3	2	-	-	-	-	-	-	-	-	-
		3. Local hosting KML sentieri e utilities	Marco	1	1	1	1	1	1	1	1	2	-	-	-	-	-	-	-	-	-

continua...

ID Item	Sprint Task	Vol.	Init.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
	4. Load testing e profiling DB.	Giacomo	3	3	3	3	3	3	3	3	3	2	2	1	1	-	-	-	-	-	-
	5. Testing	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
	6. Documentazione	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
	7. Deploy	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
22 Come Capogruppo, voglio ricevere i segnali SOS dei partecipanti e poterli validare tramite la mia dashboard, così da attivare i soccorsi solo in caso di reale necessità.	1. Modello Emergency con stati: ACTIVE, SHARED - WITH_GROUP, DISMISSED, CANCELLED_BY_SENDER	Federico	2	2	2	2	2	2	1	-	-	-	-	-	-	-	-	-	-	-	-
	2. Endpoint GET/PATCH /a-pi/v1/sessions/:id/emergencies.	Federico	3	3	3	3	3	3	2	-	-	-	-	-	-	-	-	-	-	-	-
	3. UI Registra: icona SOS, lista, dettaglio, annulla/condividi con gli altri utenti della sessione.	Federico	3	3	3	3	3	3	2	1	-	-	-	-	-	-	-	-	-	-	-
	4. Notifica capogruppo nuovo SOS (polling 8s).	Federico	2	2	2	2	2	2	1	-	-	-	-	-	-	-	-	-	-	-	-
	5. Testing	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-
	6. Documentazione	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-
	7. Deploy	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-
20 Come Capogruppo, voglio poter visualizzare la posizione GPS di tutti i partecipanti in tempo reale sulla mappa, così da monitorare la coesione del gruppo. Il tutto deve avere un'interfaccia pulita e intuitiva.	1. Endpoint GET /a-pi/v1/sessions/:id/positions (ultime coordinate).	Federico	2	2	2	2	2	2	2	1	-	-	-	-	-	-	-	-	-	-	-
	2. Integrazione Socket.io server + client Android.	Marco	4	3	3	3	3	3	3	2	-	-	-	-	-	-	-	-	-	-	-
	3. Overlay marker partecipanti su mappa Registra.	Marco	3	3	3	3	3	3	3	2	-	-	-	-	-	-	-	-	-	-	-
	5. Throttle refresh per risparmio batteria.	Federico	2	2	2	2	2	2	2	2	-	-	-	-	-	-	-	-	-	-	-
	6. Testing	Federico	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-
	7. Documentazione	Federico	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-
	8. Deploy	Federico	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-
21 Come Capogruppo, voglio poter ricevere un allarme dai componenti del gruppo (online e beacon BLE) così da gestire le emergenze e coordinare la risposta.	1. Notifica locale SOS con Poll 8s	Federico	2	2	2	2	2	2	1	-	-	-	-	-	-	-	-	-	-	-	-
	2. Invio payload SOS con coordinate.	Federico	2	2	2	2	2	2	1	-	-	-	-	-	-	-	-	-	-	-	-
	3. BLE beacon advertising post-SOS (identificativo utente/sessione, stile AirTag).	Federico	4	4	4	4	4	4	2	-	-	-	-	-	-	-	-	-	-	-	-
	4. Scanner BLE capogruppo per localizzazione prossimità utente SOS.	Federico	3	3	3	3	3	3	3	2	-	-	-	-	-	-	-	-	-	-	-
	5. UI capogruppo: allerta emergenza + indicazione distanza stimata (RSSI).	Federico	2	2	2	2	2	2	2	1	-	-	-	-	-	-	-	-	-	-	-
	6. Condivisione con gruppo dell'emergenza da parte del capogruppo	Federico	2	2	2	2	2	2	2	1	-	-	-	-	-	-	-	-	-	-	-
	7. Revoca condivisione con gruppo	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-
	8. No scanner se beaconActive==false	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-
	9. Testing	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-
	10. Documentazione	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-
	11. Deploy	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-
14 Come escursionista, voglio vedere la posizione degli altri membri del gruppo sulla mappa, così da sapere dove si trovano rispetto a me, con un'implementazione efficiente ed efficace anche a livello energetico.	1. API posizioni membri gruppo (riuso telemetria batch).	Marco	2	2	2	2	2	2	2	2	2	-	-	-	-	-	-	-	-	-	-
	2. Marker multipli + refresh efficiente su OSMdroid.	Federico	3	3	3	3	3	3	3	3	3	-	-	-	-	-	-	-	-	-	-
	3. Interval adattivo GPS/rete per risparmio energetico.	Federico	2	2	2	2	2	2	2	2	2	-	-	-	-	-	-	-	-	-	-
	4. Testing	Federico	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-
	5. Documentazione	Federico	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-
	6. Deploy	Federico	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-
37 Come sistema, voglio sincronizzare gli eventi di gamification accumulati offline tramite Event Sourcing in batch, così da garantire la consistenza del saldo Crediti Sociali.	1. Collection user_event_store (Event Sourcing).	Giacomo	3	3	3	3	3	3	3	3	3	3	2	2	1	-	-	-	-	-	-

continua...

ID Item	Sprint Task	Vol.	Init.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
	2. POST /a-pi/v1/users/:id/gamification/syt batch idempotente.	Giacomo	3	3	3	3	3	3	3	3	3	3	3	2	1	-	-	-	-	-	-
	3. Coda offline Room eventi gamification.	Giacomo	3	3	3	3	3	3	3	3	3	3	3	2	1	-	-	-	-	-	-
	4. WorkManager sync al ripristino rete.	Giacomo	2	2	2	2	2	2	2	2	2	2	2	2	2	-	-	-	-	-	-
	5. Testing	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
	6. Documentazione	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
	7. Deploy	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
26 Come escursionista che ha premuto SOS per errore, voglio poter annullare il segnale prima che venga inoltrato, così da evitare l'attivazione inutile dei soccorsi.	1. Countdown prima dell'invio (15 secondi).	Federico	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-	-	-
	2. PATCH /a-pi/v1/emergencies/:id status CANCELLED.	Federico	2	2	2	2	2	2	1	-	-	-	-	-	-	-	-	-	-	-	-
	4. Testing	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-
	5. Documentazione	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-
	6. Deploy	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-
38 Come sistema, voglio garantire che ogni evento di gamification sia processato una sola volta, così da prevenire il double-spending dei Crediti Sociali. Idempotenza	1. Campo idempotencyKey UUID su ogni evento.	Giacomo	2	2	2	2	2	2	2	2	2	2	2	1	1	-	-	-	-	-	-
	2. Unique index MongoDB (userId, idempotencyKey).	Giacomo	2	2	2	2	2	2	2	2	2	2	2	1	1	-	-	-	-	-	-
	3. Handler reject duplicate / double-spending.	Giacomo	2	2	2	2	2	2	2	2	2	2	2	1	1	-	-	-	-	-	-
	4. Testing	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
	5. Documentazione	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
	6. Deploy	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
17 Come escursionista, voglio ricevere notifiche di allerta meteo o pericoli nel percorso, così da prendere decisioni informate sulla mia sicurezza.	1. Regole alert (soglie meteo + pericoli percorso).	Giacomo	3	3	3	3	3	3	3	3	3	3	3	2	1	1	1	1	1	1	1
	2. Notifiche in-app / push escursionista.	Giacomo	3	3	3	3	3	3	3	3	3	3	3	2	1	1	1	1	1	1	1
	3. API per alert meteo	Giacomo	2	2	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1
	4. Testing	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	5. Documentazione	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	6. Deploy	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
27 Come Gestore Rifugio, voglio poter inviare notifiche push di allerta pericolo agli escursionisti nella zona, così da informarli tempestivamente di situazioni pericolose.	1. Modello DangerAlert + geofence zona rifugio.	Giacomo	3	3	3	3	3	3	3	3	3	3	3	2	1	1	1	1	1	1	1
	2. Endpoint invio alert + integrazione FCM.	Giacomo	4	4	4	4	4	4	4	4	4	4	3	3	2	2	2	2	2	2	2
	3. UI dashboard rifugio invio allerta pericolo.	Giacomo	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
	4. Testing	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	5. Documentazione	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	6. Deploy	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
16 Come escursionista, voglio poter consultare i dati di navigazione (quota, velocità, distanza percorsa) durante l'escursione, così da monitorare il mio progresso.	1. UI metriche quota/-velocità/distanza in RegistraScreen.	Federico	2	2	2	2	2	2	1	-	-	-	-	-	-	-	-	-	-	-	-
	2. Calcolo altitudine (GPS).	Federico	2	2	2	2	2	2	1	-	-	-	-	-	-	-	-	-	-	-	-
	3. Persistenza snapshot metriche in Room.	Federico	2	2	2	2	2	2	2	2	-	-	-	-	-	-	-	-	-	-	-
	4. Testing	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-
	5. Documentazione	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-
	6. Deploy	Federico	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-
45 Come escursionista, voglio vedere la difficoltà tecnica e il dislivello di una attività, così da valutare se è adatto alle mie capacità in modo chiaro sull'interfaccia grafica in app	1. Badge difficoltà/dislivello su card sessione/attività.	Giacomo	2	2	2	2	2	2	2	1	-	-	-	-	-	-	-	-	-	-	-
	2. Modello Sentier.js Backend e operazioni CRUD	Marco	2	2	2	2	2	2	1	-	-	-	-	-	-	-	-	-	-	-	-
	3. Parsing da KML	Marco	2	2	2	2	2	2	1	-	-	-	-	-	-	-	-	-	-	-	-

continua...

ID Item	Sprint Task	Vol.	Init.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
	Filtraggio backend sentieri per difficoltà, durata, dislivello	Marco	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	2. Sezione metriche in SessionDetail e Pianifica.	Giacomo	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	1	1	1
	3. Mapping difficultyLevel / elevationGain da GPX stats.	Giacomo	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	1	1
	4. Testing	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	5. Documentazione	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	6. Deploy	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
46 Come escursionista, voglio poter abbandonare una sessione di gruppo, così da gestire autonomamente la mia partecipazione.	1. Pulsante Abbandona sessione in SessionDetail.	Giacomo	2	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	2. Dialog conferma + refresh lista Le mie sessioni.	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	3. Edge case creator vs partecipante (CREATOR_CANNOT_LEAVE).	Giacomo	2	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	4. Testing	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	5. Documentazione	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	6. Deploy	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
41 Come sistema, voglio essere compatibile con dispositivi Android 9.0 o superiori, così da coprire la maggior parte del parco dispositivi degli utenti.	1. Verifica minSdk 28 e audit dipendenze.	Giacomo	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
	2. Test matrix API (emulatori/dispositivi).	Giacomo	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
	3. Fix deprecations critiche API legacy.	Giacomo	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
	4. Testing	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	5. Documentazione	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	6. Deploy	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
30 Come Gestore Rifugio, voglio poter visualizzare il conteggio delle persone presenti nel rifugio tramite sensori ottici, così da gestire la capacità in modo efficiente.	1. Integrazione MQTT telemetria sensori ottici.	Giacomo	4	4	4	4	4	4	4	4	4	4	2	1	1	1	1	1	1	1	1
	2. Endpoint occupazione rifugio in tempo reale.	Giacomo	2	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1
	3. Dashboard rifugio widget conteggio presenze.	Giacomo	2	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1
	4. Testing	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	5. Documentazione	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	6. Deploy	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
49 Come utente, voglio una schermata Profilo completa (dati personali, esperienza, obiettivi, livello e crediti sociali, foto), così da gestire la mia identità in app e vedere la mia progressione.	1. Schema backend personalInfo/experience/preferences/-goals + PATCH /users/me/* + anti-cheat	Giacomo	4	4	3	2	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	2. Onboarding profilo 3 step (skippable) + profileCompletedAt	Giacomo	3	3	3	2	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	3. ProfileScreen + ProfileV2ViewModel (sezioni, banner, navigazione edit)	Giacomo	4	4	4	4	4	4	3	1	1	1	1	1	1	1	1	1	1	1	1
	4. Avatar upload Base64 + AvatarImage + privacy gate in sessioni	Giacomo	3	3	3	3	3	3	3	2	1	1	1	1	1	1	1	1	1	1	1
	5. ProfileViewScreen read-only + indicatori anti-cheat	Giacomo	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	1	1
	6. UserProfileScreen + follow-stats + navigazione da social	Giacomo	3	3	3	3	3	3	3	3	2	1	1	1	1	1	1	1	1	1	1
	7. GET /users/me/credits + card livello in Profilo	Giacomo	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	1
	8. Testing profilo + persistenza account	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	9. Documentazione sprint2 - profilo_formazione.md	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	10. Deploy / smoke Render + build mobile	Federico	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
50 Come utente, voglio feed sociale, interazioni (follow, like, commenti), stories 24h e editor visivo, così da seguire la community e condividere le escursioni.	1. Modelli follow/comment/-likes/sharedAt + socialService backend	Giacomo	4	4	4	4	4	4	4	4	3	1	1	1	1	1	1	1	1	1	1
	2. Endpoint feed, follow, like, commenti + test Jest social	Giacomo	4	4	4	4	4	4	4	4	4	3	1	1	1	1	1	1	1	1	1
	3. SocialFeedViewModel + tab Social + FeedCard + PostDetail	Giacomo	4	4	4	4	4	4	4	4	4	3	2	2	1	1	1	1	1	1	1

continua...

ID Item	Sprint Task	Vol.	Init.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
	4. Social row avatar (anello live/story/goal) + viewed_stories Room	Giacomo	3	3	3	3	3	3	3	3	3	2	1	-	-	-	-	-	-	-	-
	5. Ricerca utenti + follow + notifiche (badge campanella)	Giacomo	3	3	3	3	3	3	3	3	3	3	3	2	1	-	-	-	-	-	-
	6. Backend story schema + route CRUD + markViewed + validazione overlay	Federico	4	4	4	4	4	4	4	4	4	4	4	3	2	-	-	-	-	-	-
	7. StoryViewer + StoryComposer + snapshot mappa OSM	Federico	4	4	4	4	4	4	4	4	4	4	4	3	2	-	-	-	-	-	-
	8. Campi editor story (overlay polyline, testo, map box overlay)	Federico	3	3	3	3	3	3	3	3	3	3	3	3	1	-	-	-	-	-	-
	9. Fix batch storie/join/mappe + delete post + pull-to-refresh Unisciti	Giacomo	3	3	3	3	3	3	3	3	3	3	3	3	1	-	-	-	-	-	-
	10. Polyline feed + sync Attività/Social	Giacomo	2	2	2	2	2	2	2	2	2	2	2	1	-	-	-	-	-	-	-
	11. Testing feed + stories + permessi	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
	12. Documentazione sprint2_ - social.md + API story	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
	13. Deploy	Federico	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
52 Come utente, voglio il modulo Formazione con quiz a risposta multipla, crediti al primo superamento e checkpoint NFC (mockup), così da imparare e progredire nei livelli.	1. Modelli QuizCategory/-Quiz/QuizAttempt + seed quizzes.json	Giacomo	4	4	4	4	4	4	4	3	1	-	-	-	-	-	-	-	-	-	-
	2. quizService submit + quizRoutes (no leak risposte su GET)	Giacomo	4	4	4	4	4	4	4	3	1	-	-	-	-	-	-	-	-	-	-
	3. Accredito crediti + CreditTransaction source quiz	Giacomo	3	3	3	3	3	3	3	3	2	-	-	-	-	-	-	-	-	-	-
	4. FormazioneScreen + FormazioneViewModel	Giacomo	3	3	3	3	3	3	3	3	3	1	-	-	-	-	-	-	-	-	-
	5. QuizScreen + feedback per domanda + risultato	Giacomo	4	4	4	4	4	4	4	4	4	3	2	2	1	-	-	-	-	-	-
	6. Auto-seed quiz al boot server	Giacomo	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-	-	-	-	-
	7. Test Jest quiz + idempotenza superamento	Marco	2	2	2	2	2	2	2	2	2	2	2	1	1	-	-	-	-	-	-
	[Mockup] Modello Totem + nfcService + endpoint scan	Giacomo	4	4	4	4	4	4	4	4	4	4	4	3	1	-	-	-	-	-	-
	[Mockup] NfcScanScreen + hardware check + feedback UX	Giacomo	3	3	3	3	3	3	3	3	3	3	3	3	2	-	-	-	-	-	-
	[Mockup] Statistiche NFC in Profilo (nfcStats)	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
	11. Testing mobile quiz + documentazione seed	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
	12. Deploy	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
54 Come utente, voglio verificare la password prima di modificare dati sensibili dell'account e poterla cambiare in sicurezza, così da proteggere le mie informazioni.	1. Backend verifyPassword + changePassword + Joi (accountRoutes)	Giacomo	3	3	3	3	3	3	3	2	1	-	-	-	-	-	-	-	-	-	-
	2. AccountEditViewModel + gate passwordVerified	Giacomo	2	2	2	2	2	2	2	2	1	1	-	-	-	-	-	-	-	-	-
	3. AccountEditScreen hub + navigazione sotto-schermate	Giacomo	2	2	2	2	2	2	2	2	1	1	-	-	-	-	-	-	-	-	-
	4. Edit PersonalInfo / Experience / Preferences / Goals	Giacomo	4	4	4	4	4	4	4	4	4	3	2	2	1	-	-	-	-	-	-
	5. Flusso cambio password (schermata dedicata)	Giacomo	2	2	2	2	2	2	2	2	2	2	2	1	1	-	-	-	-	-	-
	6. DeleteAccountScreen + conferma password (GDPR)	Giacomo	2	2	2	2	2	2	2	2	2	2	2	1	1	-	-	-	-	-	-
	7. PATCH email con re-verifica (se previsto)	Giacomo	2	2	2	2	2	2	2	2	2	2	2	2	2	-	-	-	-	-	-
	8. Testing account + password errata	Marco	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
	9. Documentazione SECURITY / account	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
	10. Deploy	Federico	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
55 Come utente, voglio un'interfaccia coerente (palette, tipografia, icone) in tutta l'app, così da avere un'esperienza professionale e leggibile.	1. Nuova palette + Theme.kt Material 3	Giacomo	4	4	4	4	4	4	4	4	4	4	4	3	1	-	-	-	-	-	-
	2. Logo e asset launcher / icona app	Giacomo	2	2	2	2	2	2	2	2	2	2	2	2	1	-	-	-	-	-	-
	3. Rifinitura Social + Home (card, effetti)	Giacomo	3	3	3	3	3	3	3	3	3	3	3	2	1	-	-	-	-	-	-
	4. Skeleton UI + Refuge Dashboard polish	Giacomo	2	2	2	2	2	2	2	2	2	2	2	2	1	-	-	-	-	-	-

continua...

ID Item	Sprint Task	Vol.	Init.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
	5. Allineamento sessione/registra/checklist al theme	Federico	3	3	3	3	3	3	3	3	3	3	3	2	1	-	-	-	-	-	-
	6. Dettagli icona e fix visivi minori	Federico	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
	7. Testing regressione visiva smoke	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
	8. Deploy	Federico	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
56 Come sistema, voglio correggere i bug emersi in Sprint 1 e durante Sprint 2, così da garantire stabilità in demo e su Render.	1. Merge conflict Render + fix deploy backend	Giacomo	3	3	2	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	2. Bug hunt security / validation / ownership (22/05)	Giacomo	4	4	4	3	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	3. Fix weather auth + Jest setup route (Marco)	Marco	3	3	3	3	2	1	-	-	-	-	-	-	-	-	-	-	-	-	-
	4. Test integrazione checklist/sentieri/admin	Marco	2	2	2	2	2	2	2	2	2	2	2	1	-	-	-	-	-	-	-
	5. Swagger / email / path normalize	Giacomo	2	2	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	6. Dialog attività corta + Registra dialog 3 opzioni	Giacomo	2	2	2	2	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	7. AVVIA tab switch + date format + merge sentieri	Federico	3	3	3	3	3	3	2	2	1	-	-	-	-	-	-	-	-	-	-
	8. Polyline visibility + activity/social/checklist fixes	Federico	4	4	4	4	4	4	4	4	4	3	2	2	1	-	-	-	-	-	-
	9. Minor bugs join/storie/icona (03-04/06)	Giacomo	2	2	2	2	2	2	2	2	2	2	2	2	1	-	-	-	-	-	-
	10. Testing regressione + documentazione TSM_PROJECT_STATE	Giacomo	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
	11. Deploy	Federico	1	1	1	1	1	1	1	1	1	1	1	1	1	-	-	-	-	-	-
TOTALE			409	409	404	380	363	352	338	314	276	222	175	145	117	83	20	20	20	20	20

Tabella 7: Sprint Backlog completo con stime iniziali ed effort rimanente per giorno.

Lo Sprint 2 Backlog completo è disponibile in *docs/SCRUM_Logs_Excel/M4_6_ActiveProductBacklog_Sprint_2.xlsx* nel repository.

Burndown Chart

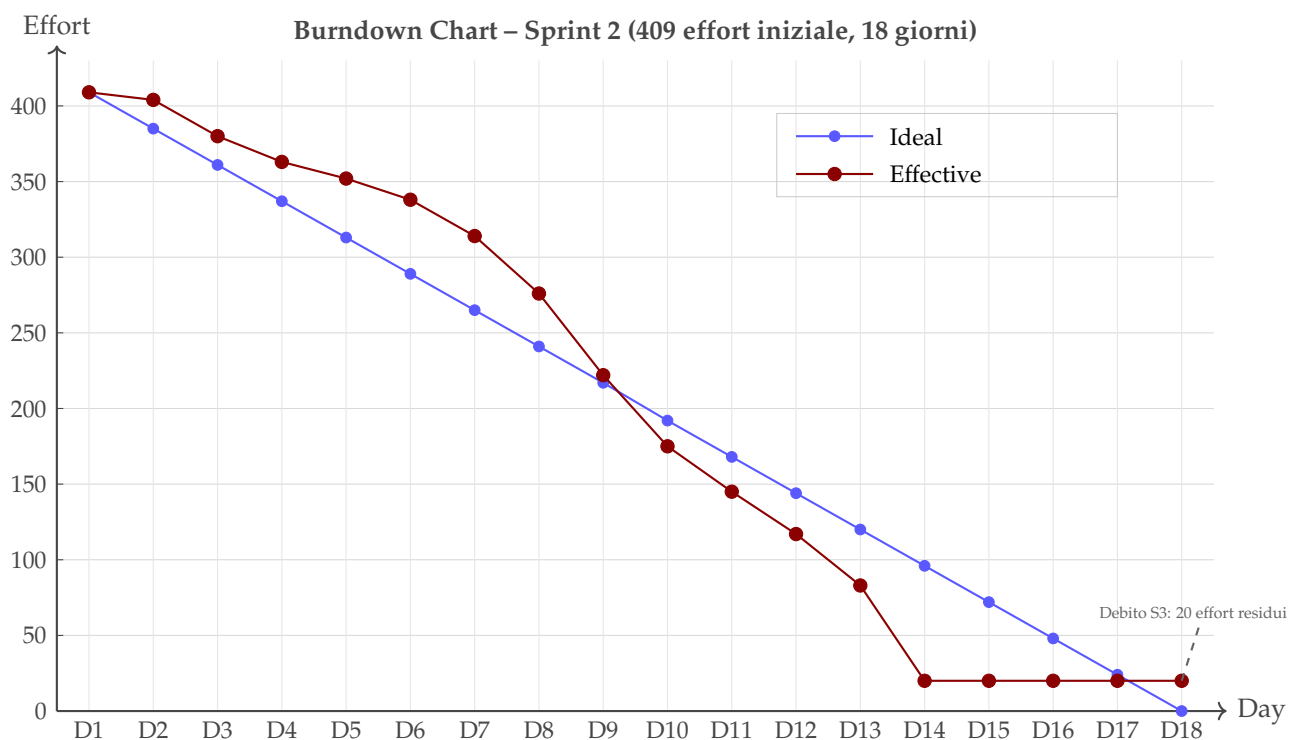


Figura 1: Burndown Chart Sprint 2. L'effort iniziale è 409. A fine sprint resta un debito di 20 unità di effort rimandato a Sprint 3 riferito alle seguenti user stories: ID17, ID27, ID30

3.3 Test Cases

In Sprint 2, in linea con l'evoluzione della Definition of Done, sono stati implementati **262 test Jest automatici** (19 suite) a coprire tutte le route principali del backend — un significativo avanzamento rispetto a Sprint 1 in cui era richiesto solo il design dei test case. La coverage misurata da Jest (*-coverage*) è del **61,7% statements / 64,1% lines** complessivi, con punte dell'82,8% sul layer middleware e del 92,2% sui modelli Mongoose; il service layer (58,3%) è il target dichiarato per Sprint 3.

La tabella seguente riporta i test case dello Sprint 1 con il risultato ottenuto durante i test. Sono stati documentati formalmente solamente questi 63 test case, ma ne sono stati implementati, come sopra menzionato, 262 per verificare la corretta implementazione delle API backend, al fine di ottimizzare i tempi di diagnostica e velocizzare la fase di debugging in seguito all'output di Jest.

TC	Output atteso	Output ottenuto
Come utente voglio poter creare un account inserendo i miei dati, così da accedere alla piattaforma usufruire dei suoi servizi.		
TC-1	HTTP 201. User stored with isVerified: false, passwordHash (bcrypt) and verificationToken. Response body contains message 'Allocazione completata. Attesa verifica email.' and user object (without passwordHash, verificationToken, __v). Verification email sent.	✓ Jest PASSED.
TC-2	HTTP 409 with body { message: 'Collisione rilevata: Email o Username già utilizzati.' }. No new user is created.	✓ Jest PASSED.
TC-3	HTTP 201. User saved with role = 'groupLeader' (schema default). Same response payload as standard success case.	✓ Jest PASSED.
TC-4	HTTP 409 with body { message: 'Collisione rilevata: Email o Username già utilizzati.' }. No new user is created.	✓ Jest PASSED.
TC-5	HTTP 500 with Mongoose validation error: Path username is required. No user is created.	✓ Jest PASSED.
TC-6	HTTP 500 with Mongoose validation error: Path email is required. No user is created.	✓ Jest PASSED.
TC-7	HTTP 500. bcrypt.hash(undefined, 10) throws. No user is created.	✓ Jest PASSED.
TC-8	HTTP 500 with Mongoose enum validation error. No user is created.	✓ Jest PASSED.
TC-9	HTTP 201 — user still created and persisted (isVerified: false). SMTP error logged server-side, does NOT propagate to client.	✓ Manual.
Come utente voglio poter effettuare login con le mie credenziali così da accedere alle funzionalità del sistema		
TC-10	Login successful. HTTP 200 with JWT token in response body.	✓ Jest PASSED.
TC-11	HTTP 401 with message 'Invalid email'.	✓ Jest PASSED.
TC-12	HTTP 401 with message 'password is invalid'.	✓ Jest PASSED.
TC-13	HTTP 403 with message 'Accesso negato. Eseguire la verifica SMTP inviata via email.'	✓ Jest PASSED.
continua...		

TC	Output atteso	Output ottenuto
TC-14	HTTP 401 with message 'Invalid email'. User.findOne({ email: " " }) returns null.	✓ Jest PASSED.
TC-15	HTTP 401 with message 'password is invalid'. bcrypt.compare("", storedHash) returns false.	✓ Jest PASSED.
Come utente autenticato voglio poter effettuare il logout dall'app		
TC-16	JWT cleared from local storage. App redirects to login screen. No further authenticated requests can be made.	✓ Manual (Android).
TC-17	App redirects to login screen. No error thrown. No API call made.	✓ Manual (Android).
Come sistema voglio mantenere attivo tracciamento GPS		
TC-18	—	✓ Manual (Android).
Come Capogruppo, voglio poter creare nuova sessione di escursione		
TC-19	HTTP 201. New HikeSession created with status: 'PLANNED', unique inviteCode in format 'TSM-XXXX', creator added to participants with role: 'groupLeader'. Response body contains full session object.	✓ Jest PASSED.
TC-20	HTTP 400 with body { error: 'routeDetails.name obbligatorio' }. No session is created.	✓ Jest PASSED.
TC-21	HTTP 409 with body { error: 'Hai una sessione attualmente in corso (tracciamento ATTIVO). Concludila prima di crearne un'altra.' }. No new session is created.	✓ Jest PASSED.
TC-22	HTTP 422 with Joi validation error. No session is created.	✓ Jest PASSED.
TC-23	HTTP 401. No session is created.	✓ Jest PASSED.
Come escursionista, voglio poter unirmi a un'escursione di gruppo inserendo il codice invito		
TC-24	HTTP 200. User added to participants with role: 'hiker'. sessionRoles updated. Response body is updated session object.	✓ Jest PASSED.
TC-25	HTTP 404 with body { error: 'Codice invito non valido' }	✓ Jest PASSED.
TC-26	HTTP 409 with body { error: 'La sessione non è più aperta' }.	✓ Jest PASSED.
TC-27	HTTP 409 with body { error: 'Sei già in questa sessione' }.	✓ Jest PASSED.
TC-28	HTTP 422 with Joi validation error.	✓ Jest PASSED.
TC-29	HTTP 409 with body { error: 'Hai una sessione attualmente in corso (tracciamento ATTIVO). Concludila prima di unirti a una nuova.' }.	✓ Jest PASSED.
Come escursionista voglio poter visualizzare le informazioni aggiornate delle sessioni a cui mi sono unito		
<i>continua...</i>		

TC	Output atteso	Output ottenuto
TC-30	HTTP 200. Array of session objects populated with username and email, sorted by meetingDate ascending. Returns [] if no sessions.	✓ Jest PASSED.
TC-31	HTTP 200. Full session object with creatorId and participants.userId populated.	✓ Jest PASSED.
TC-32	HTTP 404 with body { error: 'Sessione non trovata' }	✓ Jest PASSED.
TC-33	HTTP 400 with body { error: 'ID non valido' }	✓ Jest PASSED.
Come utente voglio poter aggiornare la sessione (data, nome, etc.)		
TC-34	HTTP 200. status updated to 'ACTIVE' and startTime set to current timestamp. Response body is updated session object.	✓ Jest PASSED.
TC-35	HTTP 200. status updated to 'COMPLETED' and endTime set to current timestamp.	✓ Jest PASSED.
TC-36	HTTP 403 with body { error: 'Solo il Capogruppo può modificare la sessione' }.	✓ Jest PASSED.
TC-37	HTTP 422 with Joi validation error.	✓ Jest PASSED.
TC-38	HTTP 422 with Joi validation error.	✓ Jest PASSED.
TC-39	HTTP 200. Specified fields updated. inviteCode NOT changed. Response body is updated session with creatorId and participants.userId populated.	✓ Jest PASSED.
TC-40	HTTP 403 with body { error: 'Solo il Capogruppo può modificare la sessione' }	✓ Jest PASSED.
Come partecipante voglio poter uscire da una sessione alla quale mi sono unito		
TC-41	HTTP 200. User removed from participants. Response body is updated session object.	✓ Jest PASSED.
TC-42	HTTP 403 with body { error: 'Il Capogruppo non può abbandonare la sessione. Eliminala se vuoi rimuoverla.' }	✓ Jest PASSED.
Come capogruppo voglio poter cancellare/eliminare una sessione		
TC-43	HTTP 200 with body { message: 'Sessione eliminata' }. Session document permanently removed from the database.	✓ Jest PASSED.
TC-44	HTTP 403 with body { error: 'Solo il Capogruppo può eliminare la sessione' }.	✓ Jest PASSED.
escursionista, voglio che il tracciamento GPS si riduca automaticamente quando sono fermo		
TC-45	—	✓ Manual (Android).
Come escursionista voglio visualizzare una checklist dell'equipaggiamento necessaria basata sull'itinerario e sulle condizioni meteo		
TC-46	—	✓ Manual (Android).
continua...		

TC	Output atteso	Output ottenuto
voglio che la mia posizione GPS venga tracciata automaticamente in background durante l'escursione		
TC-47	—	✓ Manual (Android).
escursionista, voglio che il tracciamento GPS si riduca automaticamente quando sono fermo (batteria)		
TC-48	—	✓ Manual (Android).
Come Amministratore, voglio poter gestire le registrazioni degli utenti (assegnare/modificare ruoli)		
TC-49	HTTP 200. Response body is the updated user object with role: 'rifugio'. Fields passwordHash and __v excluded.	✓ Jest PASSED.
TC-50	HTTP 500. Mongoose enum validation error. User's role not changed.	✓ Manual.
TC-51	HTTP 403 with body { message: 'Forbidden.' }. User's role not changed.	✓ Jest PASSED.
TC-52	HTTP 401 with body { message: 'No token provided.' }. Neither requireRoles nor updateUser is reached.	✓ Jest PASSED.
TC-53	HTTP 404 with body { message: 'Utente non trovato.' }.	✓ Jest PASSED.
TC-54	HTTP 400 with body { message: 'ID utente non valido.' }.	✓ Jest PASSED.
TC-55	HTTP 409 with body { message: 'Email o username già in uso.' }. Target user not modified.	✓ Jest PASSED.
TC-56	HTTP 200. Response body is the user object unchanged. passwordHash and __v excluded.	✓ Jest PASSED.
TC-57	HTTP 200 with body { message: 'Utente eliminato con successo.' }. User document permanently removed.	✓ Jest PASSED.
TC-58	HTTP 403 with body { message: 'Forbidden.' }. User not deleted.	✓ Jest PASSED.
TC-59	HTTP 404 with body { message: 'Utente non trovato.' }.	✓ Jest PASSED.
TC-60	HTTP 400 with body { message: 'ID utente non valido.' }.	✓ Jest PASSED.
Come Capogruppo, voglio poter generare un codice invito alfanumerico e un QR Code per gruppo		
TC-61	HTTP 201. Response body contains inviteCode matching regex <code>{TSM-[0-9A-F]{4}\$}</code> . Code stored uppercase.	✓ Jest PASSED.
TC-62	Both return HTTP 201. inviteCode values are different strings.	✓ Jest PASSED.
TC-63	HTTP 200. meetingLocation updated to 'Piazzale Roma'. inviteCode in response unchanged from original.	✓ Jest PASSED.

Tabella 8: Test case Sprint 1.

Suite completa: `backend/__tests__` / con 19 file per un totale di 262 test. Verifica locale 10/06/2026: `npm test` esegue tutta la suite in ~25 s di runtime Jest.

3.4 Sprint Review

La Sprint Review si è svolta venerdì 08/06/2026 con una demo live di circa 3 ore che ha coperto i seguenti flussi (verificando le effettive *novità* rispetto alla demo di Sprint 1):

1. **Onboarding 3-step e Profilo v2:** Login → rilevamento primo accesso → wizard 3-step (Dati personali birthDate/sex/peso/altezza → Esperienza outdoor: Livello CAI, Livello forma fisica, Frequenza di allenamento → Preferenze distanza/dislivello/frequenza) → tap “Salta” su uno step → verifica che `profileCompletedAt` resta null e l’app permette comunque la navigazione principale.
2. **Anti-cheat live demo:** Apertura `ProfileViewScreen` → campi con icona  (`birthDate`, `caiLevel`) → nell’app viene visualizzato “non modificabile”. **Tentativo di bypass via curl:** `PATCH /api/v1/users/me/experience body={caiLevel:T}` → **HTTP 409 LockedFieldError**. La demo ha sottolineato la difesa in profondità (UI + server).
3. **Refresh token rotation in azione:** Login → access TTL 15m + refresh TTL 30g. Simulazione token scaduto via Postman (manipolazione TTL): `TsmAuthenticator` intercetta il 401, fa refresh sincrono, ritenta la request originale – **trasparente per il ViewModel**. Demo del replay attack: riuso di un refresh già ruotato → re-login forzato.
4. **Activities libere e sync resiliente:** Registrazione attività libera senza sessione di gruppo (Capogruppo solo) → KPI strip (Distanza/Durata/Dislivello/Punti) → stop tracking → **simulazione assenza rete:** upload fallisce, attività marcata `isSynced=0`; pulsante “Risincronizza (1)” visibile in `ActivityListScreen`; tap → `enqueueImmediate(ignoreBackoff=true)` → upload riuscito; badge contatore aggiornato a 0.
5. **WAL crash-safety:** Demo del flusso “crash durante tracking”: tap AVVIA → 2 minuti di tracking GPS sintetico → kill forzato del processo da DDMS → riapertura app → verifica che la tabella `tracking_wal` contiene tutti i punti raccolti (presentazione SQL Inspector di Android Studio).
6. **Backend security live:** Apertura di `https://trento-smart-mountain-xz7u.onrender.com/api-docs` → **stress test rate limit:** 11 chiamate `POST /auth/login` consecutive → **HTTP 429** sull’undicesima. Tentativo di NoSQL injection con `$ne: null`: sanificato. Tentativo di mass-assignment `role: admin` in registrazione: **HTTP 400 Joi**.
7. **Test suite Jest:** Lancio di `npm test` in live coding → **19 suite, 262 test, tutti verdi in ~25s di runtime Jest**. Particolare attenzione ai 4 test in `discriminator.test.js` che bloccano il regress del bug critico del 26/05.
8. **Q&A e variazioni backlog:** Discussione su priorità Social Feed UI vs NFC totem per Sprint 3.

Aspetti rilevanti emersi dalla discussione post-demo:

- Il **refresh token rotation con replay detection** è stato riconosciuto come maturità oltre il livello atteso per un progetto universitario, in particolare la mutex su refresh per evitare N refresh paralleli quando più request scadono insieme.
- La gestione *trasparente* del refresh lato mobile via `Authenticator OkHttp` (zero modifiche ai `ViewModel` esistenti) è considerata un esempio virtuoso di disaccoppiamento.
- L’anti-cheat lato server ha generato discussione costruttiva sulla **difesa in profondità**: UI lock (alpha 0.6) + indicator  + server enforcement (409). Il fatto che il bug originale fosse “UI lock ma server permissivo” è stato un caso di studio efficace.
- La copertura test 0→262 è stata accolta come investimento strategico (ogni nuovo bug fix richiede ora un test di regressione associato).
- **Cold start Render free tier** (~60–100s) ha causato la prima richiesta della demo: il team ha chiarito che è accettabile per la fase universitaria, mitigato in produzione da paid tier.

3.5 Product Backlog Refinement

A seguito della Sprint Review, il team ha aggiornato il Product Backlog con le seguenti nuove User Story per Sprint 3, derivate dal debito tecnico residuo e dai piani già scritti durante Sprint 2:

Priorità	User Story / Motivazione
Must	NFC totem sul campo – Posa dei tag NFC fisici a vetta/rifugio e pilot con rifugio partner. App e backend già operativi (NfcScanScreen + POST /api/v1/nfc/scan con reward Social Credits): resta solo l'hardware.
Should	Modulo rifiuti: persistenza data entry – Evoluzione dell'MVP read-only ADR-002 (simulatore già in produzione): registrazione giacenze stagionali per categoria, report costi mensili, benchmark anonimo tra rifugi della piattaforma.
Must	Refactor qualità area rifugista – Pass completo di design system (gerarchia, glass, motion) su dashboard IoT, profilo rifugio e simulatore rifiuti, per allinearli alla cura grafica del resto dell'app.
Should	SOS propagazione BLE Mesh (US-11) – Beacon BLE single-hop (advertising + scanner RSSI) e backend POST /api/v1/emergencies già in produzione; resta la propagazione mesh multi-hop con firma ECC e $\text{hopCount} \leq 10$ (richiede prototipazione hardware).
Could	OAuth Google login – Login social con account Google, in alternativa a email/password.
Could	Socket.io live tracking – Posizioni real-time del gruppo nella dashboard capogruppo (dipendenza già installata).
Could	Sentry integration – Logging strutturato + crash reporting (oggi solo console.log).
Could	WorkManager mobile – Sync robusto anche dopo OS kill (oggi richiede processo vivo); migrazione da coroutine loop a WorkManager periodic.
Could	CMS web admin per quiz – Oggi seed JSON in repo; auspicabile editor web per amministratori non-tecnici.
Must	Mappa avanzata + routing in-app – Editor percorsi personalizzati direttamente nell'app (senza app di terze parti come intermediario), calcolo percorso tra due punti su sentieri CAI, esportazione e importazione .gpx. Obiettivo: portare in-house ciò che oggi viene delegato a Komoot/Wikiloc.
Must	Supporto wearable (Wear OS / BLE) – Integrazione smartwatch per tracking biometrico (battito, passi); interfaccia ottimizzata watch-only per mountain mode; collegamento dati fitness alle statistiche hike.
Must	Sistema suggestion anti-overtourism – Algoritmo raccomandazione percorsi basato sull'affluenza in tempo reale (conteggio utenti attivi per sentiero). Suggerisce alternative ai percorsi sovraffollati. Dashboard analytics per enti parco e guide alpine.
Must	Rifugi "VIP" – monetizzazione – Sistema a tempo limitato: i rifugi partner acquistano finestre temporali con moltiplicatori di crediti per gli utenti (es. 2x per 2 settimane). Revenue model B2B per la piattaforma. Admin panel per gestione campagne.
continua...	

Priorità	User Story / Motivazione
Must	Partnership attrezzatura sportiva + referral – Integrazione brand (Salewa, Arc'teryx, Mammut) per checklist dinamiche con link referral specifici per prodotto e condizioni meteo. Revenue sharing: visibilità in-app in cambio di commissione sulle vendite. Checklist generate automaticamente da difficoltà sentiero e previsioni.
Could	Dashboard rifugista avanzata – Upgrade schermata rifugista: gestione prenotazioni posti, messaggistica con escursionisti in avvicinamento, bollettino meteo locale personalizzato, telemetria IoT macchinari (MQTT) per manutenzione predittiva.
Won't (questa release)	Modalità gara quiz – Timer per domanda + leaderboard. Discusso ma posticipato post-deliverable.

Tabella 9: Product Backlog Refinement: nuove User Story emerse durante lo Sprint 2 e priorità per lo Sprint 3.

Variazioni al Product Backlog esistente:

- US-11 (SOS) downgrade Must→Should: l'UI è in app, ma la propagazione BLE Mesh richiede prototipazione hardware che esce dal perimetro Sprint 3.
- US-20 (Dashboard real-time capogruppo) downgrade Should→Could in attesa di Socket.io implementation.
- US-28 (Telemetria IoT macchinari) confermata Could; richiede gateway Python operativo, fuori scope universitario.
- **Anti-cheat caiLevel rivisitazione:** discussione sull'opportunità di permettere "upgrade" del livello (T→EEA) post-attività verificate. Posticipato a Sprint 3 con piano dedicato.

3.6 Sprint Retrospective

Cosa ha funzionato

- ✓ **Audit a 2 passi formalizzato:** L'introduzione del criterio 6 nella Definition of Done (analisi statica CWE + cross-validation post-fix) ha intercettato *prima del merge* la regressione potenziale "FIELD_LOCKED: dopo il return null" che avrebbe fatto fallire 2 test anti-cheat. Si conferma l'efficacia del processo.
- ✓ **Test-driven bugfix:** I 3 bug critici di Sprint 2 (vedi sotto) sono stati fixati *con* test di regressione contestuali, non a posteriori. La cartella discriminator.test.js (4 test) è un esempio: blocca la regressione del bug più insidioso dello sprint.
- ✓ **Global error mapper:** Il pattern BUSINESS_ERROR_MAP ha rimosso 35+ blocchi if (err.message === ...) dalle 6 route principali. Le route ora sono molto più leggibili (95% next(err) puro) e i codici di errore sono *machine-readable* per il client mobile (consente UX localizzata).
- ✓ **Mongoose discriminator alignment:** Aver scoperto e formalizzato il pattern "mai User.findByIdAndUpdate per campi Hiker" ha eliminato un'intera classe di bug silenti. Il criterio 11 nella DoD lo rende strutturale.
- ✓ **Refresh token rotation trasparente lato mobile:** L'uso dell'Authenticator OkHttp ha permesso di introdurre la rotazione senza modificare nessun ViewModel. La mutex su refresh evita N refresh paralleli quando più request scadono insieme.
- ✓ **Render auto-deploy su UI:** Il feedback rapido sul comportamento "production-like" ha abbreviato il ciclo di debug rispetto a Sprint 1 (dove tutto era locale).
- ✓ **Acquisizione e persistenza locale dei dati geografici:** La scelta di scaricare i file KML dei sentieri dal portale ufficiale della SAT Trentino, versionarli su GitHub e importare i relativi metadati su MongoDB Atlas si è rivelata vincente. Persistere i dati in cloud anziché interrogare API di terze parti (*che non sono accessibili a terzi*) ad ogni richiesta ha permesso di ottimizzare le performance, ridurre i tempi di risposta e, di conseguenza, ridurre il consumo della batteria dell'applicazione Android, evitando chiamate di rete ripetute e non necessarie.
- ✓ **Social Feed e sistema Storie 24h:** La creazione di un'entità Story reale con TTL index MongoDB (scadenza automatica 24h) anziché derivare le storie dai post si è rivelata architetturealmente robusta. L'editor drag&drop con gesture di transform (scala, rotazione, traslazione) porta la condivisione a un livello paragonabile ad app commerciali.
- ✓ **ADR-001 – Redesign ciclo vita sessione:** Il refactoring da modello ibrido a participationState state machine ortogonale all'approvazione ha eliminato il *ghost member bug*: il capogruppo può terminare la sessione anche se un partecipante non ha chiamato /complete. Il force-close con auto-finalizzazione risolve strutturalmente un intero scenario di blocco.
- ✓ **Approvazione partecipanti (pending/accepted):** Il workflow join con pending state, approvazione da capogruppo o partecipante accettato, e idempotenza (richiesta già inviata) ha introdotto un layer di controllo sulle sessioni di gruppo senza degradare l'UX.
- ✓ **UI Premium (glass morphism, aurora, animazioni):** L'interfaccia con glass cards, aurora particles animate, shimmer sul badge credits, frecce direzionali animate sulla traccia GPX e double-tap like ha portato l'aspetto dell'app a un livello paragonabile ad applicazioni commerciali come Komoot e Strava prese come leader del settore da attaccare.

Cosa non ha funzionato – Bug critici identificati in Sprint 2

3 Bug Critici – batch fix del 26/05/2026

Bug S2-C1 – Discriminator persistence (silent drop): Tutti i write su campi del sotto-schema Hiker (personalInfo, experience, preferences, weeklyGoals, socialCredits, nfcStats.*) venivano fatti via `User.findByIdAndUpdate`. Lo strict mode di Mongoose applicato al modello base scartava silenziosamente l'`$set/$inc`: la response tornava 200 OK ma il DB non veniva toccato. Bug invisibile perché:

- Le response 200 facevano pensare a un save riuscito.
- I successivi `findById` ritornavano i dati esistenti (la projection MongoDB funziona indipendentemente dal modello).
- Le ViewModel scoped-to-Activity mostravano gli ultimi valori in memoria.

✓ *Fix applicato 26/05: tutti i write su campi discriminator usano ora `Hiker.findByIdAndUpdate` (o il modello corretto via `lookup role`). 4 service toccati.*

Test contract: `discriminator.test.js` (4 test) + `account.test.js` (13 test).

Bug S2-C2 – Anti-cheat enforcement server-side mancante: Frontend mostrava lucchetto  su `birthDate` e `caiLevel` dopo prima impostazione, ma chiunque poteva aggirare con curl/Postman e abbassare il livello CAI per farmare crediti facili. Caso classico di “client-side only validation” visto in audit OWASP.

✓ *Fix applicato 26/05: `updatePersonalInfo` e `updateExperience` rilanciano*

`LockedFieldError` → HTTP 409 se il campo è già impostato. Mobile parsa il campo `message` del body di errore per UX leggibile.

Bug S2-C3 – JWT expiry 1h insufficiente per requisito offline 3gg: La RNF di TSM richiede uso offline fino a 3 giorni, ma JWT expiry era di 1h. Dopo il primo blackout di rete, il token scadeva e l'utente era forzato a re-login – impossibile in montagna senza copertura.

✓ *Fix applicato 26/05 + ulteriore rifinitura SS2: flusso login + refresh con access TTL 15m e refresh TTL 30d (`TsmAuthenticator` trasparente lato mobile). Il deep link post-verifica email emette ancora un JWT standalone a TTL lungo (`JWT_EXPIRES_IN`, default 7d) per il primo bootstrap offline.*

Debito tecnico residuo (per Sprint 3)

Voci marcate ✓ sono state risolte durante Sprint 2; voci marcate ◦ restano debito aperto per Sprint 3.

- ✓ **Pattern Repository violato in 4 ViewModel** (residuo S1): refactor parziale completato in Sprint 2 con l'estrazione di `TrackingPersistenceRepository` e `SessionCommandRepository` dal `RegistraViewModel`. Restano 2 ViewModel da rifattorizzare (`ActivityListViewModel`, `SessionJoinViewModel`).
- ✓ **meetingDate String→Date** (residuo S1): migrazione completata con backward compat 100%; script di backfill in `backend/migrations/2026-05-26-meetingDate-string-to-date.js`.
- ✓ **Room migration helper pattern** (nuovo S2): `TsmMigrations.kt` come single source of truth per le migration esplicite. Pattern documentato per i futuri bump (schema Room v8 in produzione, WAL introdotto in v5).
- **Logging strutturato + Sentry**: oggi solo `console.log`. Sprint 3 introdurrà Pino + Sentry.
- **Audit trail per azioni admin**: chi ha eliminato chi, chi ha cambiato ruoli; oggi non tracciato.
- **Rate limit con store Redis**: oggi in-memory, OK per single-instance Render. In prod Sprint 3 con multi-instance richiederà Redis.
- **CI gate su npm audit**: la verifica locale del 10/06/2026 riporta 0 vulnerabilities; Sprint 3 introdurrà GitHub Actions con gate bloccante per mantenere il vincolo nel tempo.
- **Coverage Jest service layer**: coverage complessiva 61,7% statements (middleware 82,8%, models 92,2%, routes 65,5%, services 58,3%). Hot spot da coprire in Sprint 3: `adminService` (8,8%), `checklistService` (4,3%), `refugeService` (7,0%), `sentieroService` (9,4%), `weatherService` (32,8%) e le nuove route di approvazione partecipanti in `hikeSessionRoutes` (oggi 44,8% route-level).
- **WorkManager mobile**: WorkManager è già usato per l'upload differito delle emergenze; resta da estendere il pattern al sync generale delle attività, che oggi usa un loop coroutine attivo finché il processo è in vita.
- **Recovery dialog post-crash dalla WAL**: l'infrastruttura WAL è in produzione, manca la UX di recovery alla riapertura.

Bug noti a fine Sprint 2 (dalla code-review del 10/06)

Tre difetti emersi dal field testing degli ultimi giorni di sprint hanno costituito il primo input del backlog di Sprint 3: la sessione di code-review dedicata del 10/06 ne ha confermato le cause radice e prodotto i fix.

- ✗ **B-01 — Zoom mappa bloccato a livello troppo distante**. Su alcune schermate mappa lo zoom-in 'rimbalzava' indietro impedendo di raggiungere il dettaglio del sentiero (regressione introdotta con l'unificazione delle 'mappe zoomabili').
 ✓ *Fix applicabile proposta del 10/06: causa radice confermata — l'auto-fit (`zoomToBoundingBox`) era keyed sull'identità della lista punti, quindi ogni recomposition Compose ri-fittava la mappa al bounding box annullando lo zoom dell'utente. Ora il `LaunchedEffect` è keyed su un fingerprint del contenuto (`routeKey/contentKey`); in aggiunta l'overlay frecce direzionali campiona la traccia a max 200 punti per frame (prima proiettava migliaia di punti GPX a ogni draw, rendendo scattoso il pinch-zoom su tracce lunghe).*
- ✗ **B-02 — Badge "NFC ATTIVO" nel profilo non riflette lo stato reale**. Il badge restava verde anche con NFC disattivato dalle impostazioni di sistema; la lettura dello stato non era real-time.
 ✓ *Fix applicato il 10/06: badge a 3 stati (ATTIVO / DISATTIVATO / NON DISPONIBILE) legato allo stato reattivo `nfcEnabled` anziché alla sola presenza del chip; `BroadcastReceiver` su `ACTION_ADAPTER_STATE_CHANGED` collegato al rendering, così lo stato si aggiorna in tempo reale anche attivando NFC dalla tendina rapida senza lasciare la schermata (stesso pattern applicabile a `NfcScanScreen`).*

- × **B-03 — Chiusura sessione incoerente tra schermate.** Il tasto “Termina/Arresta” nella schermata Unisciti aveva un comportamento diverso dallo stesso tasto nel dettaglio sessione: i due flussi seguivano code path divergenti e la state machine participationState di ADR-001 era applicata solo parzialmente lato UI.

✓ *Fix applicato 10/06: i due flussi convergono su un'unica azione a livello ViewModel con **dialog di conferma esplicito** (“la sessione verrà conclusa per TUTTI i partecipanti — azione irreversibile”) prima del force-close ADR-001 con auto-finalize dei membri live. Lato backend la guardia FORBIDDEN_NOT_LEADER resta l'enforcement autoritativo: solo capogruppo o leader effettivo (failover) possono chiudere per tutti.*

Action items per Sprint 3

1. **Completare US-17, US-27, US-30** (Must – debito tecnico): alert meteo push, notifiche pericolo rifugio, monitoraggio affollamento IoT.
2. **Bug-fix batch da code-review** (Must): B-01 zoom mappa, B-02 badge NFC real-time, B-03 convergenza flussi “Termina” — ✓ *wip dal 10/06* (vedi *Bug noti* sopra).
3. **Routing e mappa avanzata in-app** (Must): editor percorsi personalizzati, calcolo path su sentieri CAI, import/export .gpx senza dipendenza da terzi.
4. **NFC totem check-in + Social Credits** (Must): check-in a vetta, reward crediti, hardware NFC reader integrato.
5. **Sistema suggestion anti-overtourism** (Should): algoritmo affluenza percorsi, suggerimenti alternativi, dashboard analytics.
6. **Rifugi VIP + monetizzazione** (Should): moltiplicatori crediti a tempo, admin panel campagne, revenue model B2B.
7. **Partnership attrezzatura sportiva + referral** (Could): checklist dinamiche con link branded, commissioni su vendite.
8. **Supporto wearable Wear OS** (Could): tracking biometrico, interfaccia watch-only.
9. **Sentry + Pino logging + CI gate** (Should): osservabilità produzione, npm audit bloccante.
10. **WorkManager mobile** (Could): estensione del pattern già usato per le emergenze al sync robusto delle attività post OS-kill.
11. **Dashboard rifugista avanzata** (Could): prenotazioni, messaggistica, telemetria IoT MQTT, modulo gestione rifiuti e calcolo costi di trasporto (ADR-002 — *MVP simulatore già abbozzato il 10/06*, resta la persistenza del data entry stagionale).

Dinamiche di team e pratiche Agile

Lo Sprint 2 ha confermato e raffinato l'approccio *parzialmente asincrono* dello Sprint 1, con due cambiamenti significativi:

1. **Sync call bisettimanale dedicata a verifica e triage** (istituzionalizzata dalla retrospective Sprint 1). Il batch fix del 26/05 (3 bug critici risolti contestualmente) è il risultato più tangibile di questa pratica.
2. **Audit-first development**: ogni feature non-banale viene preceduta da un audit di sicurezza (CWE-classified) sui file da toccare. Questo ha permesso di intercettare il bug “FIELD_LOCKED: dopo return null” come regressione di un fix precedente prima del merge.

Lezione appresa: l'investimento in **test automatici + audit a 2 passi** ha più che ripagato l'effort iniziale (262 test prodotti) intercettando bug che in Sprint 1 sarebbero passati silenti fino in produzione. Il pattern verrà esteso a Sprint 3 con coverage anche del service layer.

4 Visione Sprint 3

4.1 Obiettivi e roadmap

Lo Sprint 3 si articola su due binari paralleli: (a) **completamento del debito tecnico** accumulato in Sprint 2 (US-17/27/30, dipendenze hardware/IoT) e (b) **evoluzione del prodotto** verso funzionalità di alto valore per l'utente finale.

Roadmap tecnica

1. **Saldo debito tecnico (Must — 20 SP).** US-17 (alert meteo push via FCM), US-27 (notifica pericolo dal rifugio), US-30 (monitoraggio affollamento IoT). Richiede gateway Python/MQTT operativo e sensori fisici al rifugio partner. Dipendenza esterna: accordo con Rifugio Lago di Tovel per prototipo pilota.
2. **Routing avanzato e percorsi personalizzati in-app (Must).** Editor grafico su mappa per creare itinerari su sentieri CAI, calcolo profilo altimetrico, export/import .gpx nativo. Obiettivo: eliminare la dipendenza da Komoot/Wikiloc per la pianificazione pre-hike. Stack previsto: OpenRouteService API (gratuito, open-source) + OSMdroid overlay layer.
3. **NFC totem check-in + crediti sociali (Must).** Tag NFC installato a cima/rifugio; l'app legge l'ID totem e assegna crediti sociali verificati. Backend già predisposto (/api/v1/nfc/scan), hardware reader NFC in fase di acquisto.
4. **Sistema anti-overtourism con suggestion engine (Should).** Algoritmo che analizza l'affluenza storica e in real-time sui sentieri e suggerisce percorsi alternativi meno affollati. Dashboard analitica per l'amministratore e per i rifugisti. Beneficio: differenziazione competitiva + impatto ambientale positivo. I dati di affluenza alimentano anche il modulo rifiuti (ADR-002): il *free-riding* dei turisti giornalieri pesa per ~20% della massa rifiuti dei rifugi.
5. **Supporto wearable Wear OS (Should).** Companion app ottimizzata per smartwatch: avvio/-stop sessione, tracking biometrico (FC, passi), alert SOS da polso. Stack: Wear OS 3.x + Health Services API + Data Layer API.
6. **Sentry + logging strutturato + CI gate (Should).** Integrazione Sentry SDK per crash reporting in produzione; logging strutturato con Pino (JSON su Render); gate npm audit bloccante in CI. estensione WorkManager al sync attività post OS-kill (oggi già operativo per l'upload emergenze).

Roadmap business

- A. **Rifugi VIP e moltiplicatori di crediti (B2B — Should).** I rifugi partner possono attivare campagne a tempo (es. “questo weekend: 2x crediti al Rifugio X”). Revenue model: abbonamento mensile SaaS per il rifugio + revenue sharing sugli acquisti in-app. Implementazione: admin panel campagne, colonna multiplierActive nel modello Refuge, endpoint POST `/api/v1/admin/campaigns`.
- B. **Partnership attrezzatura tecnica sportiva (B2B — Could).** Collaborazione con brand di attrezzatura alpinistica (es. Mammut, Salewa) per: (i) checklist dinamiche personalizzate in base al percorso/difficoltà con link branded; (ii) referral link con commissione (3–8%) su ogni acquisto tracciato. Vantaggio per l’utente: checklist personalizzata + accesso a offerte esclusive. Implementazione: modello EquipmentPartner, endpoint catalogo, deep link al negozio partner.
- C. **Dashboard rifugista avanzata (B2B — Could).** Upgrade dall’interfaccia rifugista attuale (solo gestione account) a una piattaforma gestionale: prenotazioni posti letto, messaggistica con escursionisti, telemetria IoT (se gateway attivo), report presenze mensili e **modulo gestione rifiuti & logistica** (vedi ADR-002 sotto): censimento dei rifiuti prodotti per categoria e calcolo dei costi di trasporto a valle (elicottero, teleferica, mezzi). Obiettivo: fidelizzare i rifugi come partner strutturali, non solo come punti di check-in.

ADR-002 (Accepted — MVP implementato) — Modulo gestione rifiuti e logistica per rifugi

Status: Accepted (MVP read-only implementato il 10/06/2026) **Data:** 10/06/2026 **Deciders:** team + rifugista pilota

Contesto. La gestione dei rifiuti in quota è il costo operativo più critico per i rifugi off-grid. Il team ha quantificato il problema in un elaborato dedicato di Operations Strategy & Reverse Logistics² su un network di ~146 strutture (~110 escursionistiche, ~40 alpinistiche off-grid, di cui 9–11 servite esclusivamente da elicottero). I numeri chiave: *Logistics Drag* del 4,5% del fatturato lordo ($\approx 1,125$ €/pasto) che erode il 15–20% dell’EBIT; costo logistico unitario $c_{kg} \approx 2,25$ €/kg contro un benchmark a valle di 0,15–0,30 €/kg (scostamento 10–20×), modellato come

$$C_{kg} = \frac{C_{fix} + c_{var} \cdot t_{flight}}{P \cdot S_t}$$

con payload utile dell’H125 ridotto a 800 kg a 2.600 m s.l.m. (–33% per *density altitude*) e coefficiente di saturazione del carico attuale $S_t \approx 0,55$ (target To-Be: $S_t \geq 0,90$, massa outbound –30/50%). Il team dispone inoltre di un **simulatore web già funzionante**³ che implementa il modello: bilancio di massa stagionale su 6 categorie merceologiche (organico, plastica, vetro, metalli, cartone, indifferenziato — ciascuna con densità apparente e capacità di stoccaggio), pre-trattamento con riduzione percentuale di massa/volume, confronto tra 4 vettori outbound (elicottero, drone cargo, teleferica, mezzo terrestre) e verifica dei vincoli normativi di compliance (stoccaggio max 30 m³, giacenza max 90 giorni).

Decisione. Integrare nella dashboard rifugista un modulo *Rifiuti & Logistica* che porti il modello del simulatore dentro TSM e lo estenda con persistenza dati. La sinergia è strutturale: lo stream *Data-Driven Pull Logistics* dell’elaborato individua come collo di bottiglia proprio l’**assenza di dati in tempo reale sulle giacenze** — il gap che TSM può colmare con il monitoraggio IoT/MQTT già in roadmap (US-30) e con i dati di affluenza del sistema anti-overtourism: l’elaborato quantifica il *free-riding* dei turisti giornalieri in ~20% della massa rifiuti, con rapporto ospiti/visitatori 1:30. Decisivo il dato di adozione: **il 92% delle strutture non ha oggi alcun sistema di monitoraggio**, e la roadmap implementativa 2026–2028 dell’elaborato pone come Wave 0 proprio la costruzione della baseline dati (pilot IoT Smart Bins su 10 rifugi + una stagione di KPI) — il ruolo naturale di TSM come layer digitale del network.

²Gestione Rifiuti in Alta Quota — Associazione Rifugi Trentino, Challenge 2, elaborato OGA 2026, gruppo ID-22, consegna 22/05/2026 (allegato alla consegna). I tre componenti del team TSM fanno parte del gruppo autore.

³Simulatore Gestione Rifiuti — Rifugi Alpini: <https://biasio.github.io/simulatore-rifiuti-in-quota/>

Opzioni considerate:

- **Opzione A — Link al simulatore web esterno.** Complessità bassa, zero sviluppo backend; ma UX frammentata (il rifugista esce dall'app), nessun dato persistito, nessuna reportistica né benchmark, nessuna integrazione con la telemetria IoT.
- **Opzione B — Modulo nativo backend + app (scelta).** Complessità media: nuovo modello dati (WasteRecord: categoria, peso, data; TransportVector: mezzo, payload, costo fisso, costo variabile, fattore di riempimento), endpoint `/api/v1/refuges/:id/waste` con Joi + rate limit + `requireRoles(refuge)`, schermata dedicata nella dashboard. Benefici: storico per stagione, report costi mensili, alert di saturazione stoccaggio, benchmark anonimo tra rifugi della piattaforma, base dati per ottimizzazioni logistiche di rete (rotte *Milk Run* multi-rifugio promosse dall'elaborato).

Conseguenze. Diventa più facile: giustificare il valore B2B della piattaforma verso i rifugisti (funzionalità gestionale concreta con ROI quantificato, non solo vetrina) e candidare TSM come layer digitale della roadmap implementativa 2026–2028 dell'elaborato. Diventa più oneroso: il perimetro mobile cresce di una schermata complessa; da rivisitare a valle del feedback del rifugista pilota.

Action items: ✓ (1) portare le formule del simulatore in un service backend dedicato (`wasteService`); ✓ (2) modellare le 6 categorie merceologiche e i 4 vettori come dati di configurazione; ✓ (3a) MVP read-only (simulazione senza persistenza), ○ (3b) data entry stagionale con persistenza; ○ (4) collegare gli alert di compliance (30 m^3 , 90 gg, S_i) al sistema notifiche TSM.

Implementazione (10/06/2026). L'opzione B è stata realizzata in versione MVP read-only: `wasteService.js` implementa il bilancio di massa stagionale sulle 6 categorie merceologiche + frazione grigliato, il pre-trattamento con riduzione massa/volume, gli alert di compliance art. 185-bis (stoccaggio per categoria, 30 m^3 , 90 gg) e il confronto costi dei 4 vettori outbound con indicazione del vettore più economico. Endpoint: GET `/api/v1/refuge/waste/config` e POST `/api/v1/refuge/waste/simulate`, protetti da JWT + rate limit + `requireRoles(rifugio, admin)` + Joi (`wasteSimulationSchema`). 8 test Jest dedicati con cross-check sui numeri dell'elaborato OGA (scenario di riferimento: 2 rotazioni elicottero → 2,00 €/kg); coverage del service 93,4%. Lato mobile la schermata `WasteSimulatorScreen` è integrata nella dashboard rifugista (form parametri stagionali, KPI massa/volume/riduzione, alert compliance, tabella costi per vettore).

Principio guida Sprint 3. Mentre Sprint 1 ha costruito le fondamenta e Sprint 2 ha consolidato il processo e la sicurezza, Sprint 3 trasforma TSM in un **ecosistema**: non più solo un'app per escursionisti, ma una piattaforma che genera valore per utenti, rifugi, brand e territorio. La scelta di sviluppare componenti di monetizzazione (VIP, referral) risponde a un obiettivo concreto: rendere il prodotto **sostenibile** oltre la tesi e a valore aggiunto per tutte le entità che via accedono.

5 Sezione Finale

5.1 Diagramma del Deploy Finale

Il diagramma seguente rappresenta l'architettura di deploy finale di TSM con tutti i componenti interni ed esterni, evidenziando le interazioni di sincronizzazione e i servizi terzi integrati.

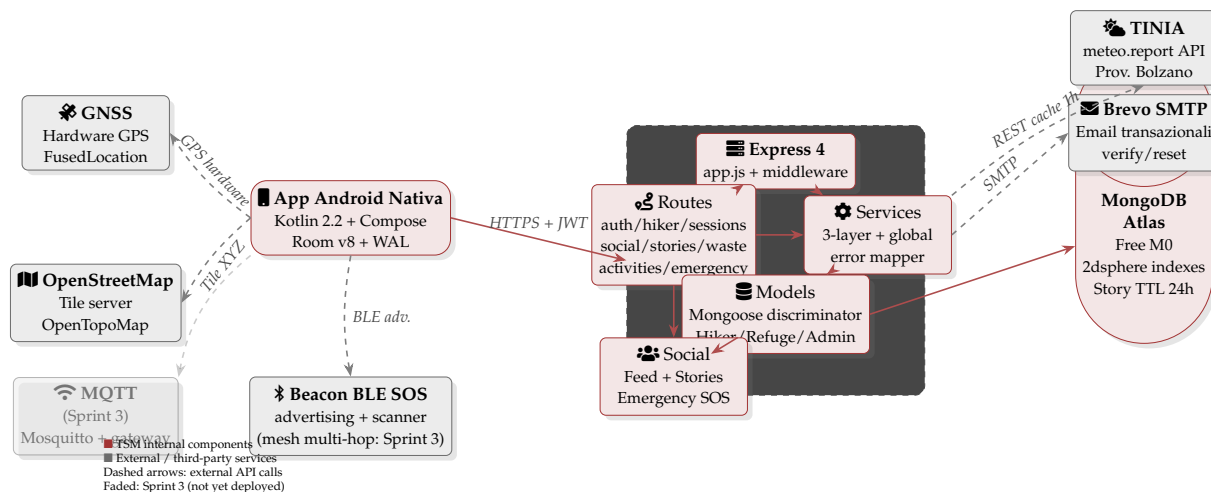


Figura 2: Diagramma deploy finale di Trento Smart Mountain a fine Sprint 2. Componenti interni: app Android nativa (Kotlin/Compose), backend Express 4 su Render, MongoDB Atlas (con TTL index 24h per Stories). Routes copre auth/hiker/sessions/social/stories/emergency/waste; servizi esterni: TINIA meteo, Brevo SMTP, OSM tile server, hardware GNSS. Il beacon BLE SOS (advertising + scanner) è operativo; il componente in trasparenza (MQTT) è placeholder per Sprint 3.

Componenti del deploy

- **Mobile (client):** APK Android nativo distribuito sideload (Debug); production-ready per Google Play Internal Testing (futuro). Comunica con il backend via HTTPS + JWT Bearer; con TsmAuthenticator per refresh rotation trasparente.
- **Backend (Render Free):** servizio Node.js 18 + Express 4 deployato su `trento-smart-mountain-xz7u.onrender.com`. Auto-deploy da branch UI. Limitazioni: single instance, cold start ~60–100 s dopo inattività.
- **Database (MongoDB Atlas Free M0):** cluster shared M0 con 512 MB storage; indici 2dsphere su location, activity.startPoint; indice composto (status, meetingDate) su hikesessions; TTL su refreshtokens (30g). **Nuovo Sprint 2:** TTL index expiresAt su collection stories (scadenza automatica a 24h).
- **TINIA / meteo.report:** API meteo della Provincia Autonoma di Bolzano, con cache 1 h su MongoDB e refresh on-demand admin-only.
- **Brevo SMTP:** email transazionali (verifica registrazione, reset password).
- **OpenStreetMap + OpenTopoMap:** tile server pubblici usati da OSMdroid per la mappa offline-friendly.
- **Hardware GNSS:** GPS del dispositivo Android via FusedLocationProviderClient; ACCESS_BACKGROUND_LOCATION + WAKE_LOCK per tracking continuo.
- **Beacon BLE SOS:** advertising BLE post-SOS (identificativo utente/sessione, stile AirTag) + scanner di prossimità del capogruppo con stima distanza RSSI — operativo in app. La propagazione mesh multi-hop resta per Sprint 3.
- **Sprint 3 (placeholder):** MQTT (gateway Python per IoT macchinari rifugio) pianificato ma non ancora deployato.

5.2 Stack Tecnologico

Lista completa di librerie, framework e pacchetti utilizzati nello sviluppo di TSM, divisi per tier.

Backend (Node.js)

Pacchetto	Versione	Scopo
node	18+	Runtime JavaScript server-side.
express	4	Framework HTTP.
mongoose	8	ODM MongoDB con discriminator support.
jsonwebtoken	9	Generazione e verifica JWT (access + refresh).
bcrypt	6	Hashing password con cost adattivo.
joi	17	Validazione schemi request body / query.
helmet	8	Security HTTP headers (CSP, HSTS).
express-rate-limit	7	Rate limit a 6 livelli (global, login, register, password reset, authenticated, write).
express-mongo-sanitize	2	Anti NoSQL injection.
hpp	0.2	HTTP Parameter Pollution protection.
cors	2	CORS con allow-list.
nodemailer	8	SMTP client legacy; l'invio Brevo corrente usa API HTTPS con fetch.
swagger-ui-express	5	UI Swagger su /api-docs.
swagger-autogen	2	Generazione automatica swagger-output.json.
jest	30	Test framework.
supertest	7	HTTP integration testing.
mongodb-memory-server	10	MongoDB in-memory per i test Jest.
dotenv	16	Caricamento env vars.
mqtt	5	Client MQTT predisposto per integrazioni IoT rifugi.
xml2js	0.6	Parsing XML/KML per dati geografici e import sentieri.

Tabella 10: Stack backend: dipendenze chiave del package.json.

Mobile (Android nativo Kotlin)

Pacchetto	Versione	Scopo
kotlin	2.2.10	Linguaggio principale.
gradle	9.4.1	Build system (wrapper).
compose-bom	2024.12.01	UI declarativa Jetpack Compose.
material3	1.3.x	Design system Material You.
navigation-compose	2.8.x	Navigazione tra schermate.
room	2.8.4	Database locale SQLite (schema v5 con WAL).
retrofit	2.11	Client HTTP REST.
okhttp	4.12	HTTP stack (+ Authenticator per refresh).
gson	2.11	Serializzazione JSON.
security-crypto	1.1.0	EncryptedSharedPreferences per JWT.
play-services-location	21.x	FusedLocationProviderClient (GPS).
osmdroid	6.1.x	Mappa offline-friendly + OpenTopoMap.
zxing	3.5	Generazione QR codici invito.
coil-compose	2.x	Caricamento immagini async.
coroutines-android	1.9	Async + StateFlow.
work-runtime-ktx	2.10	WorkManager per upload differito emergenze e futura estensione al sync attività.
junit	4.13	Unit test (modulo).

Tabella 11: Stack mobile: dipendenze chiave di mobile/app/build.gradle.kts.

Infrastruttura / DevOps

- **MongoDB Atlas** (Free Tier M0, cluster shared) – database production.
- **Render** (Free Tier) – hosting backend Node.js con auto-deploy GitHub.
- **Docker Compose** – ambiente di sviluppo locale (MongoDB + Mosquitto MQTT placeholder).
- **GitHub** – VCS, code review, issue tracking, branch protection rules.
- **Swagger / SwaggerHub** – API documentation, alternativa ad Apiary.
- **Brevo** – transactional email provider (300 email/giorno free).
- **TINIA / meteo.report** – API meteo gratuita PAB.

5.3 Conclusioni

Il Milestone 4 chiude un ciclo Agile SCRUM completo di Trento Smart Mountain, articolato su 2 sprint, di cui ~650 ore di lavoro di team nel solo Sprint 2. I numeri salienti dello **Sprint 2**:

- **78 SP pianificati, 71 completati (91%);** 7 SP in debito tecnico per Sprint 3 (US-17 alert meteo, US-27 allerta pericolo push, US-30 monitoraggio affollamento – dipendenze hardware/IoT esterno).
- **262 test Jest verdi** (19 suite), eseguibili in ~25 s di runtime Jest (`npm test`, verifica locale 10/06/2026).
- **3 bug critici identificati e fixati in-sprint** (batch del 26/05): discriminator persistence, anti-cheat server-side, JWT expiry insufficiente.
- **50+ codici di errore** centralizzati in `BUSINESS_ERROR_MAP`, rimossi 35+ blocchi `if (err.message === ...)` dalle 6 route principali.
- **Refresh token rotation con replay detection** trasparente lato mobile (`TsmAuthenticator OkHttp` con mutex).
- **Room v8 con WAL crash-safety** per i punti GPS durante tracking attivo (WAL introdotto in v5).
- **Security stack OWASP-compliant:** helmet, mongo-sanitize, hpp, rate limit 6 livelli, Joi (o schema equivalente) sugli endpoint mutanti critici.
- **ADR-002 MVP rifiuti & logistica** implementato in chiusura sprint: simulatore stagionale backend + schermata dashboard rifugista, con cross-check sui dati dell'elaborato OGA.

Risultato strategico. Più che la quantità di feature, lo Sprint 2 ha consolidato il *processo*: la Definition of Done estesa (audit a 2 passi + test automatici + discriminator alignment) ha intercettato *strutturalmente* una classe intera di bug silenti che in Sprint 1 erano sfuggiti fino all'audit di fine sprint. Il team chiude il deliverable con una piattaforma robusta e una base di test che protegge le evoluzioni di Sprint 3.

Lezione di processo. L'investimento iniziale in audit-first development e test-driven bugfix ha più che ripagato l'effort. Il batch del 26/05 (3 bug critici risolti contestualmente con 17 test di regressione) è l'esempio più tangibile: senza il criterio 6 della DoD, il bug "discriminator persistence" sarebbe rimasto silente in produzione (l'API ritornava 200 OK!) finché un utente non avesse notato che i suoi dati non venivano mai salvati. La pratica verrà estesa al service layer in Sprint 3.

Prossimi passi (Sprint 3). Sprint 3 si articola su tre priorità principali (dettaglio in § 4): (1) **saldo debito tecnico** US-17/27/30 (alert meteo push, notifiche pericolo, IoT affollamento) non completabile in Sprint 2 per dipendenze hardware; (2) **mappa avanzata e routing in-app** con editor itinerari su sentieri CAI, check-in NFC a vetta e sistema anti-overtourism; (3) **piattaforma B2B** con rifugi VIP a moltiplicatori di crediti, partnership brand sportivi con referral link, e dashboard gestionale rifugista con modulo rifiuti & logistica (ADR-002, MVP simulatore già operativo: resta la persistenza del data entry stagionale). Infine, osservabilità di produzione con Sentry + logging strutturato + CI gate `npm audit`.

Note del gruppo.

Come gruppo abbiamo scelto di andare oltre i requisiti d'esame, non per dimostrare qualcosa, ma perché credevamo nel prodotto che stavamo costruendo. La decisione di sviluppare un'app Android, più complessa rispetto a una web app, ma necessaria per garantire performance e accessibilità reali, nasce da questa stessa motivazione. L'intelligenza artificiale, di cui il gruppo ha fatto uso come descritto nella sezione B, ha contribuito a rendere questo percorso sostenibile senza comprometterne la qualità. Alla fine del secondo sprint, il risultato a nostro avviso, è più di un semplice mockup, più di un semplice progetto universitario: è un inizio, un affacciarsi al mondo della tecnologia e decidere di farne parte gettando delle basi solide per continuare a sviluppare il prodotto in un futuro prossimo.

A - API implementate Sprint 1 + Sprint 2

I principali endpoint sono documentati in Swagger (`swagger-output.json` nel repository; UI su `/api-docs`; documentazione integrale su SwaggerHub). La tabella seguente riporta il **subset principale** Sprint 1+2; per il catalogo completo (sentieri, crediti, badge, sfide, checklist, live tracking) fare riferimento a Swagger.

Metodo	Route	Auth	Descrizione
Auth (Sprint 1 + Sprint 2 extensions)			
POST	<code>/auth/register/hiker</code>	No	Registrazione hiker (rate limit 20/h, Joi).
POST	<code>/auth/register/refuge</code>	No	Registrazione rifugio (rate limit 20/h, Joi flat schema).
POST	<code>/auth/login</code>	No	Login; ritorna <code>accessToken</code> , <code>refreshToken</code> , <code>refreshExpiresAt</code> , <code>alias token</code> (backward compat). ★ S2
POST	<code>/auth/refresh</code>	No (token in body)	Refresh token rotation con replay detection; revoca family su replay.
★ S2			
POST	<code>/auth/logout</code>	No (refreshToken in body)	Revoca refresh token (idempotente). ★ S2
GET	<code>/auth/verify/:token</code>	No	Verifica email → deep link <code>tsm://</code> .
POST	<code>/auth/forgot-password</code>	No	Reset password (Brevo, rate limit 5/h).
GET / POST	<code>/auth/reset-password/:token</code>	No	Form HTML + JSON, monouso 1h.
Profilo / Account v2 (Sprint 2)			
GET	<code>/hikers/:id</code>	JWT	Profilo utente (discriminator-aware); shim legacy GET <code>/users/:id</code> .
PATCH	<code>/api/v1/users/me/personal-info</code>	JWT	Update <code>personalInfo</code> ; lock su <code>birthDate</code> . ★ S2
PATCH	<code>/api/v1/users/me/experience</code>	JWT	Update <code>experience</code> ; lock su <code>caiLevel</code> . ★ S2
PATCH	<code>/api/v1/users/me/preferences</code>	JWT	Update preferences (difficoltà, durata, dislivello). ★ S2
continua...			

Metodo	Route	Auth	Descrizione
PATCH	/api/v1/users/me/goals	JWT	Update weeklyGoals (km, dislivello). ★ S2
POST	/api/v1/users/me/profile-comp	JWT	Marca onboarding completato (profileCompletedAt). ★ S2
POST	/api/v1/users/change-password	JWT	Cambio password con vecchia password.
★ S2			
POST	/api/v1/users/me/verify-passw	JWT	Verifica password prima di modifiche sensibili. ★ S2
GET	/api/v1/users/me/weekly-stats	JWT	Statistiche settimanali aggregate. ★ S2
Sessions			
POST	/api/v1/sessions	JWT	Crea sessione (GPX stats + invite code TSM-XXXX).
GET	/api/v1/sessions/my	JWT	Sessioni dell'utente (creator + partecipante).
GET	/api/v1/sessions/stats?year=	JWT	Statistiche aggregate annuali (HikeSession + Activity). ★ S2
GET	/api/v1/sessions/:id	JWT	Dettaglio sessione populated.
POST	/api/v1/sessions/join	JWT	Join con codice TSM-XXXX; il partecipante entra in stato pending. ★ S2
POST	/api/v1/sessions/:id/particip	JWT	Approva richiesta di join (capogruppo o partecipante accettato). ★ S2
POST	/api/v1/sessions/:id/particip	JWT	Rifiuta richiesta di join pendente. ★ S2
DELETE	/api/v1/sessions/:id/particip	JWT (creator)	Rimozione definitiva partecipante (ban per la sessione). ★ S2
POST	/api/v1/sessions/:id/leave	JWT	Abbandona sessione (non-creator).
DELETE	/api/v1/sessions/:id	JWT (creator)	Elimina sessione.
PATCH	/api/v1/sessions/:id	JWT (creator)	Modifica sessione.
PATCH	/api/v1/sessions/:id/status	JWT (creator)	Lifecycle PLANNED → ACTIVE → COMPLETED.
continua...			

Metodo	Route	Auth	Descrizione
PATCH	/api/v1/sessions/:id/complete	JWT	Conclusione individuale ADR-001 (participationState = finished); la sessione passa a COMPLETED quando tutti gli accettati hanno concluso. ★ S2
POST	/api/v1/sessions/:id/close	JWT (leader)	Force-close del capogruppo con auto-finalize dei membri live (ADR-001); 403 FORBIDDEN_NOT_LEADER. ★ S2
DELETE	/api/v1/sessions/:id/from-act:	JWT	Nasconde una sessione COMPLETED dalle proprie attività (per-utente). ★ S2
POST	/api/v1/sessions/:id/live-loc:	JWT	Upload posizione live del partecipante (polling capogruppo). ★ S2
GET	/api/v1/sessions/:id/live-loc:	JWT	Fetch posizioni live dei partecipanti accettati. ★ S2
Activities libere (Sprint 2)			
POST	/api/v1/activities	JWT	Crea attività libera personale. ★ S2
GET	/api/v1/activities	JWT	Lista attività per utente. ★ S2
GET	/api/v1/activities/:id	JWT (owner)	Dettaglio attività. ★ S2
DELETE	/api/v1/activities/:id	JWT (owner)	Elimina attività. ★ S2
Weather			
GET	/weather/locations/nearby	JWT	Stazioni meteo vicine (2dsphere).
GET	/weather/locations/search	JWT	Ricerca stazioni per nome.
GET	/weather/forecast/:externalId	JWT	Forecast 3h+24h (cache 1h).
POST	/weather/seed	JWT+admin	Seed towns + POI da TINIA. Fix S1-C2
POST	/weather/forecast/:id/refresh	JWT+admin	Force refresh forecast. Fix S1-C2
Social Feed & Follow (Sprint 2)			
GET	/api/v1/users/search	JWT	Ricerca utenti per username/nome.
continua...			

Metodo	Route	Auth	Descrizione
POST	/api/v1/users/:id/follow	JWT	Segui utente (+ notifica al target).
DELETE	/api/v1/users/:id/follow	JWT	Smetti di seguire.
GET	/api/v1/users/me/feed	JWT	Feed sociale paginato (post propri + seguiti).
GET	/api/v1/users/me/social-row	JWT	Stato anelli avatar (storie attive, sessioni live).
GET	/api/v1/users/me/weekly-leade:	JWT	Classifica settimanale crediti tra seguiti.
GET	/api/v1/users/:id/posts	JWT	Post pubblicati di un profilo (privacy-gated).
GET	/api/v1/users/me/notification:	JWT	Notifiche (follow, like, commenti, approvazioni); varianti unread-count, read, DELETE.
POST	/api/v1/activities/:id/share	JWT	Condivide l'attività come post nel feed (DELETE per ritirarla).
POST	/api/v1/activities/:id/like	JWT	Like al post (DELETE per rimuoverlo).
POST	/api/v1/activities/:id/commen:	JWT	Commenta il post (Joi; GET lista, DELETE proprio commento).
Storie 24h (Sprint 2)			
POST	/api/v1/stories	JWT	Crea storia (media Base64 image/video, caption, overlay editor); scadenza automatica via TTL index 24h.
GET	/api/v1/stories/user/:userId	JWT	Storie attive (non scadute) di un autore, privacy-gated.
GET	/api/v1/stories/:id	JWT	Dettaglio singola storia.
POST	/api/v1/stories/:id/view	JWT	Marca la storia come vista (idempotente).
DELETE	/api/v1/stories/:id	JWT (autore)	Elimina la storia (403 per non-owner).
Emergency SOS (Sprint 2 backend)			
POST	/api/v1/emergencies	JWT	Crea segnalazione SOS con posizione GPS (Joi).
<i>continua...</i>			

Metodo	Route	Auth	Descrizione
GET	/api/v1/emergencies/:id	JWT	Stato della segnalazione.
PATCH	/api/v1/emergencies/:id	JWT	Aggiorna/annulla la segnalazione (US-26).
GET	/api/v1/sessions/:id/emergenc:	JWT	Emergenze attive della sessione (polling capogruppo 8 s).
Rifugio: Dashboard IoT, Rifiuti & Logistica (ADR-002), Bacheca			
GET	/api/v1/refuge/dashboard	JWT+rifugio	Telemetria sensori + edge nodes BLE + passaggi del rifugio.  S2
GET	/api/v1/refuge/waste/config	JWT+rifugio	Categorie merceologiche, vettori e limiti normativi (ADR-002).  S2
POST	/api/v1/refuge/waste/simulate	JWT+rifugio	Simulazione stagionale: bilancio massa, alert compliance, costi per vettore (Joi).  S2
GET	/api/v1/board	JWT	Feed bacheca rifugi (filtri type/activeOnly).  S2
POST	/api/v1/board	JWT+rifugio	Crea post in bacheca (PATCH/DELETE per autore o admin).
 S2			
NFC totem (Sprint 2)			
POST	/api/v1/nfc/scan	JWT	Registra scan NFC totem; \$inc nfcStats.scansCount.  S2
POST	/api/v1/nfc/admin/totem	JWT+admin	Crea totem NFC con tagId univoco.  S2
Quiz (Sprint 2 backend + UI mobile)			
GET	/api/v1/quiz/categories	JWT	Lista categorie quiz disponibili (auto-seed idempotente).  S2
GET	/api/v1/quiz/categories/:slug,	JWT	Quiz di una categoria.  S2
GET	/api/v1/quiz/categories/:slug,	JWT	Prossimo quiz disponibile nella categoria.  S2
GET	/api/v1/quiz/:id	JWT	Dettaglio quiz con domande.  S2
continua...			

Metodo	Route	Auth	Descrizione
POST	/api/v1/quiz/:id/submit	JWT	Submit risposte e calcolo crediti (idempotente al primo superamento). ★ S2

Tabella 12: Principali endpoint API Sprint 1 + 2 ...

B Utilizzo Intelligenza Artificiale nello Sviluppo

In continuità con il deliverable D3, lo sviluppo dello Sprint 2 è stato affiancato da strumenti di intelligenza artificiale generativa (Gemini, Claude Code), utilizzati come supporto alla progettazione, alla stesura del codice, all’audit di sicurezza e alla documentazione. Tale scelta non ha sostituito le decisioni architetturali né la responsabilità del team sul prodotto finale: ogni output prodotto dall’AI è stato **revisionato, integrato e validato manualmente**, in particolare nelle aree security-sensitive (refresh token rotation, anti-cheat server-side, audit del bug “discriminator persistence”). Tutti i test di regressione sono stati progettati per fissare i contratti scoperti durante l’audit, indipendentemente dall’origine dell’implementazione.

L’IA è stata particolarmente utile in diverse aree:

1. **Audit di sicurezza a 2 passi:** analisi statica CWE-classified su file modificati prima del merge.
2. **Refactor a rischio:** estrazione di `TrackingPersistenceRepository` e `SessionCommandRepository` dal `RegistraViewModel` (547→501 righe) eseguita in worktree isolata.
3. **Test coverage espansione:** dalla generazione iniziale di 60 test all’estensione a 262.
4. **Ingegnerizzazione architettura backend:** consultazione per l’ingegnerizzazione efficiente ed efficace delle collections, in particolare l’integrazione della checklist dinamica all’interno della collection `Hikesession`.
5. **Consultazione pre sviluppo:** durante il secondo sprint ci sono state diverse idee che rendevano l’implementazione delle user stories presenti più solide. Come nel caso della user story sulla selezione dei sentieri nell’app, sono stati integrati i filtri per raggruppare i sentieri e rendere la fase di pianificazione più accessibile e intuitiva. L’AI è stata utilizzata per consolidare le nostre idee prima di partire con l’implementazione vera e propria.

C Schermate Android

C.1 Nuova palette e pivot grafico



Figura 3: Nuovo Logo Trento Smart Mountain

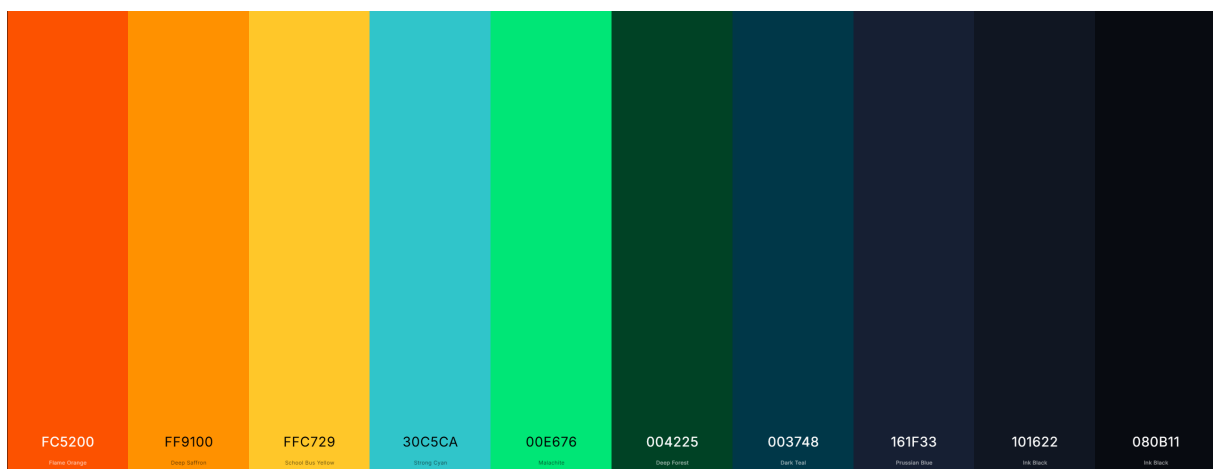


Figura 4: Palette primaria dell'applicazione